



JoBoss

AI-Powered Job Application Platform

Use Cases & UML Sequence Diagrams

Document Type: Software Requirements Specification — Feature Catalogue

Version: 1.0 | **Date:** June 2026

Author: JoBoss Development Team

33 Features

Full-Stack React + Python

AWS Serverless

AI / Bedrock

Stripe Payments

Chrome Extension

Table of Contents

F-01	User Registration & Authentication	F-11	Profile Image Upload
F-02	Onboarding Flow	F-12	Subscription & Stripe Payment
F-03	Job Discovery & Swipe Interface	F-13	Daily Swipe Quota Enforcement
F-04	Job Filtering & Matching Algorithm	F-14	Admin Dashboard & Statistics
F-05	Swipe Recording (Like / Pass)	F-15	Admin User Management
F-06	Undo Last Swipe	F-16	Admin Job Management
F-07	Application Tracking	F-17	Job Importer (Telegram)
F-08	Auto-Apply Automation	F-18	Job Status Monitoring
F-09	AI Resume Tailoring	F-19	Chrome Extension Auto-Fill
F-10	Resume Upload & Management	F-20	Email Notification (SES)
F-21	Personal Dashboard	F-22	Preference Mismatch Warning
F-23	Edit Profile & Change Password	F-24	Account Suspension Screen
F-25	Password Reset Flow	F-26	Discovery Mode (30-min Window)
F-27	Match Score Breakdown Modal	F-28	Show All Jobs Toggle
F-29	GPS Location Auto-Detection	F-30	Chrome Ext. — Direct Login
F-31	Chrome Ext. — SSO Auth Bridge	F-32	Chrome Ext. — Tailored CV Selector
F-33	Chrome Ext. — Floating Fill Button		



Authentication & User Profile

F-01 → F-03 & F-10 → F-12

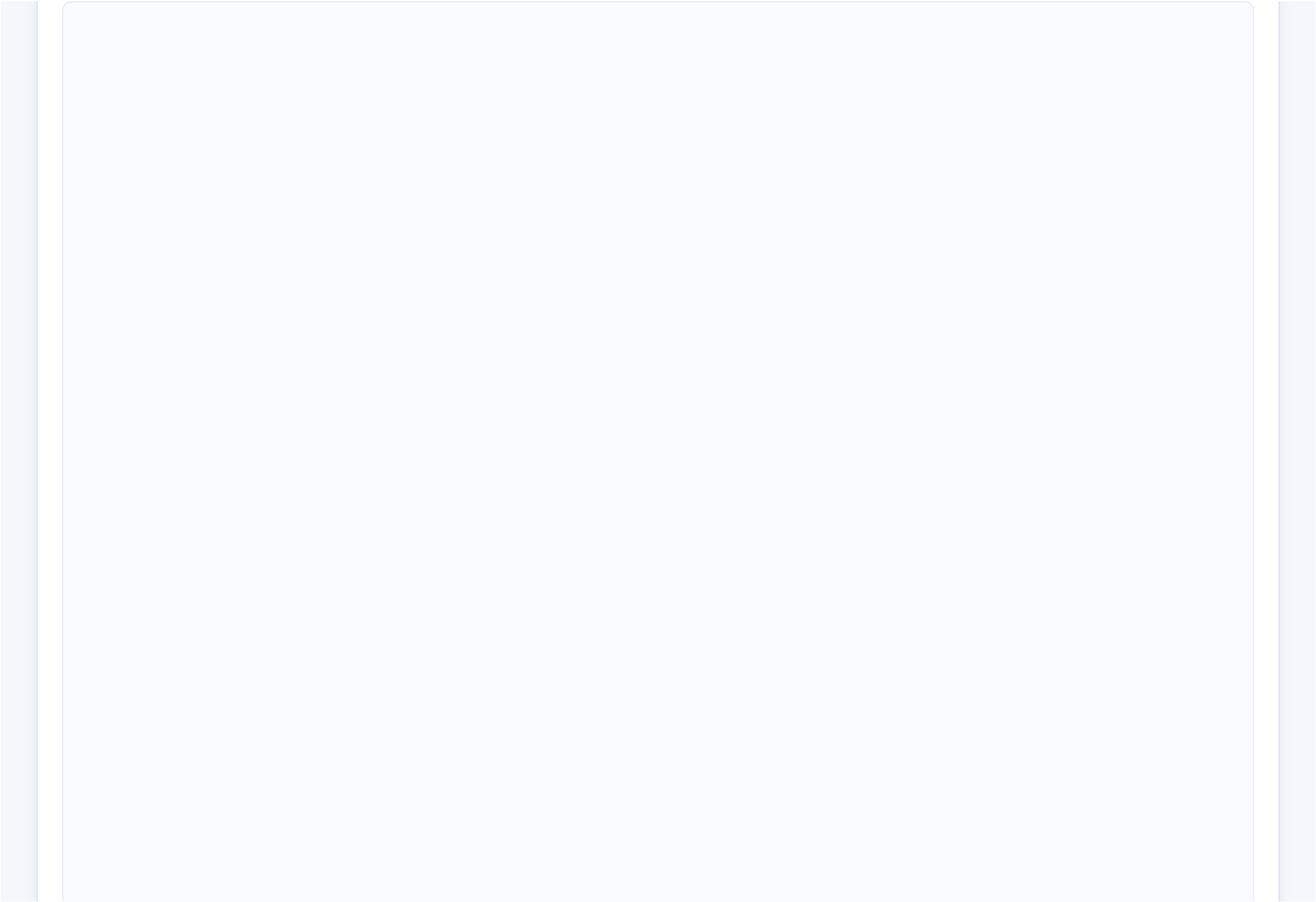
F-01

User Registration & Authentication

Actor: Visitor / User AWS Cognito API Gateway All Tiers

Users authenticate via AWS Cognito using OAuth social login (Google/GitHub) or email/password. A JWT access token is issued and attached to every subsequent API request. On first login, users are redirected to the onboarding flow. Suspended accounts receive a blocked-account error and cannot proceed.

Implements: `src/pages/LoginPage.jsx` `src/aws-exports.js` `lambdas/users/handler.py` `lambdas/admin/handler.py`



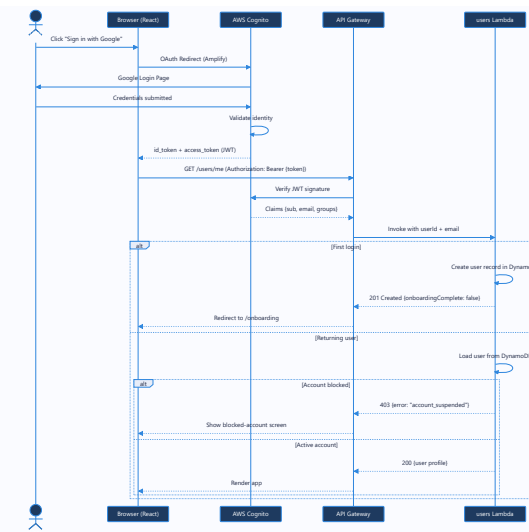


Fig F-01: User Registration & Authentication Flow

F-02

Onboarding Flow (5-Step Setup)

Actor: New User

Onboarding Wizard

AI CV Analysis

Bedrock

New users are guided through a 5-step modal wizard: (1) Plan selection, (2) Resume upload with AI keyword extraction, (3) Role & category preferences, (4) Location & distance preferences, (5) Confirmation. After completing all steps, onboardingComplete is set to true and the user lands on the swipe page.

Implements: `src/pages/OnboardingPage.jsx` `lambdas/users/handler.py` `lambdas/uploads/handler.py` `lambdas/ai/handler.py`

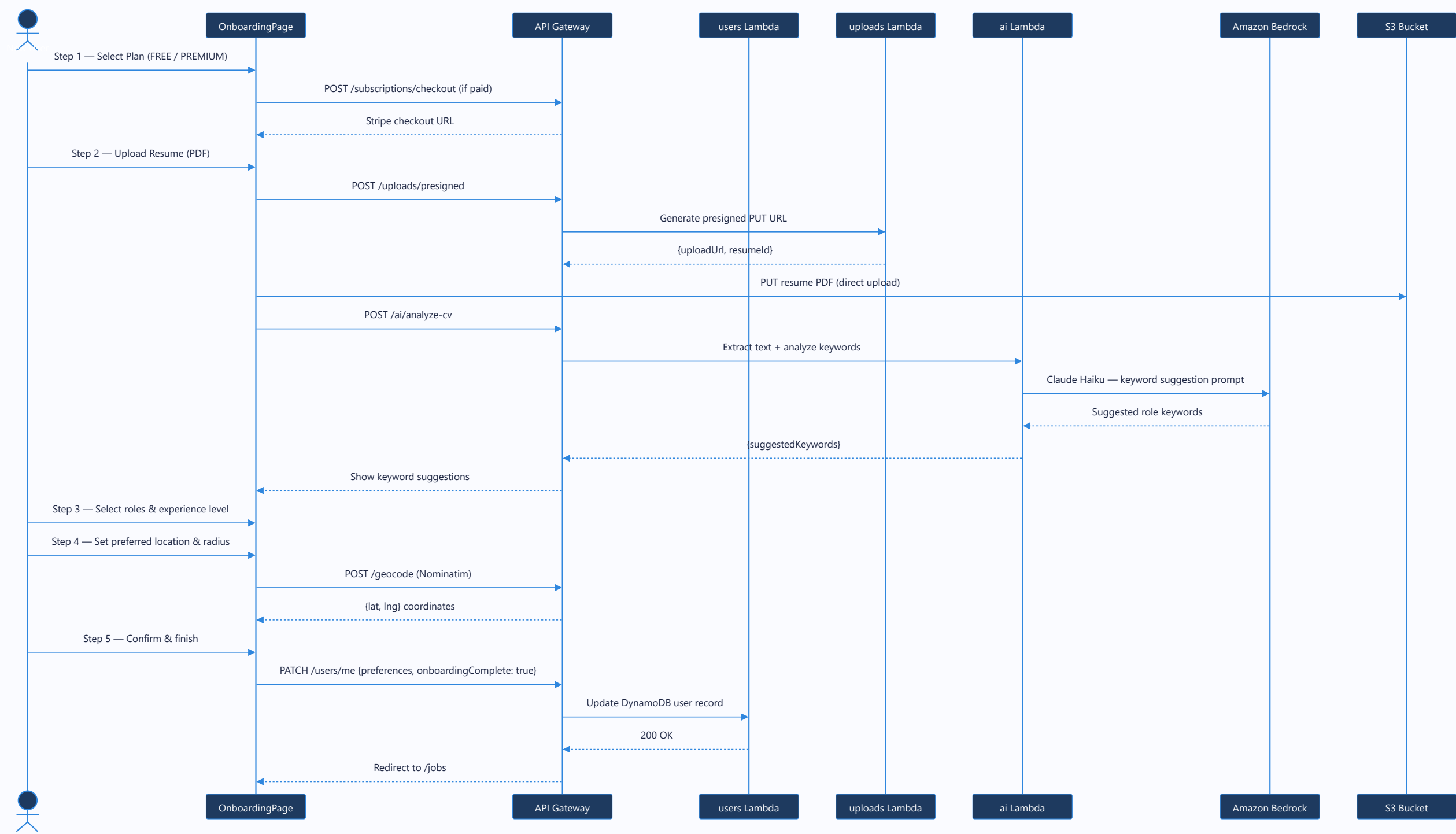


Fig F-02: 5-Step Onboarding Wizard Flow

②

Job Discovery & Swiping

F-03 → F-07

F-03

Job Discovery & Swipe Interface

Actor: Authenticated User

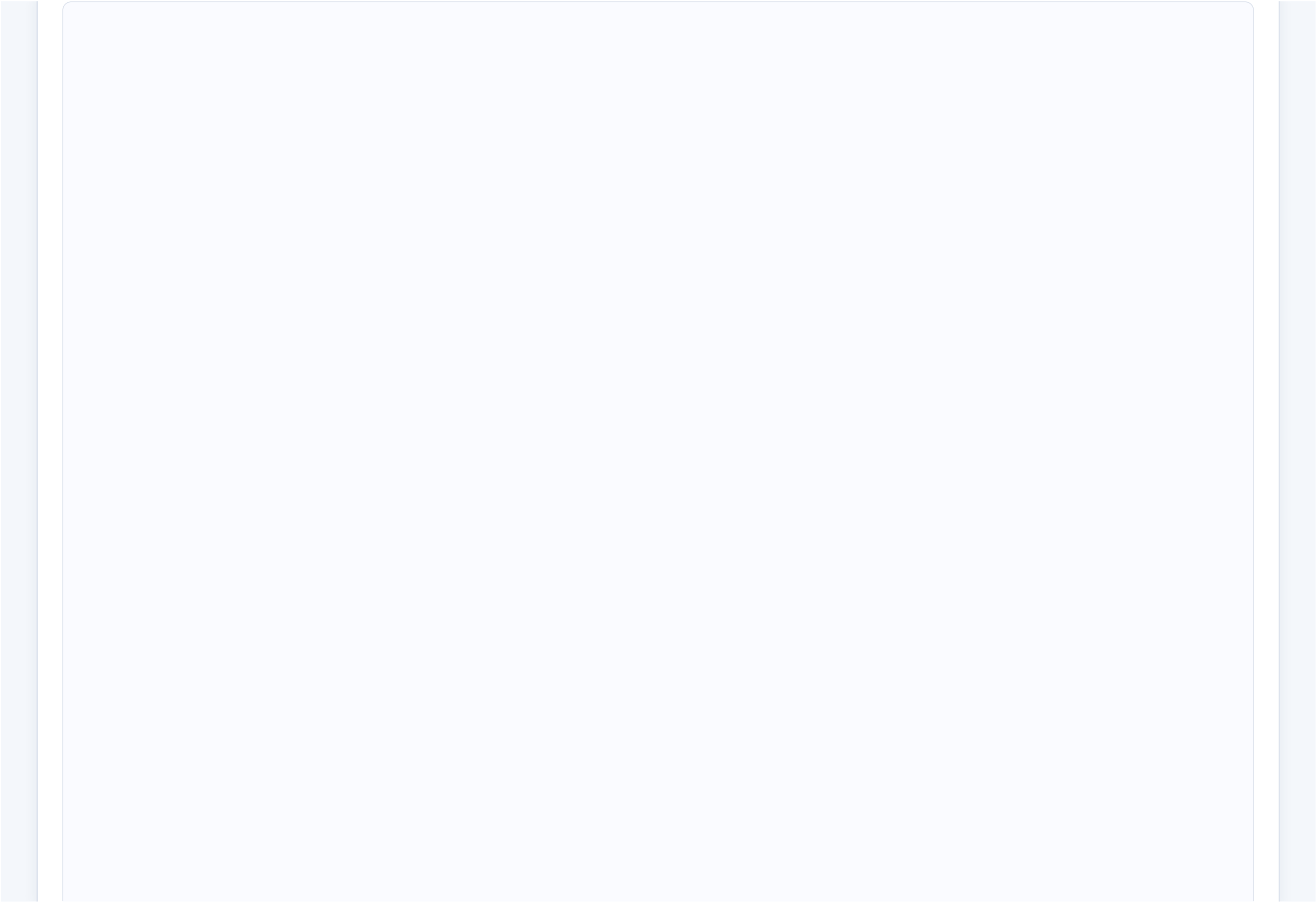
SwipePage

DynamoDB

All Tiers

The main discovery screen loads a personalised stack of job cards. Each card shows the company logo, job title, location, and salary range. Users can swipe right (LIKE) or left (PASS), or open a full-detail modal. Card animations are powered by Framer Motion with drag gestures. Jobs already swiped are excluded from future loads.

Implements: `src/pages/SwipePage.jsx` `lambdas/jobs/handler.py` `lambdas/swipes/handler.py`



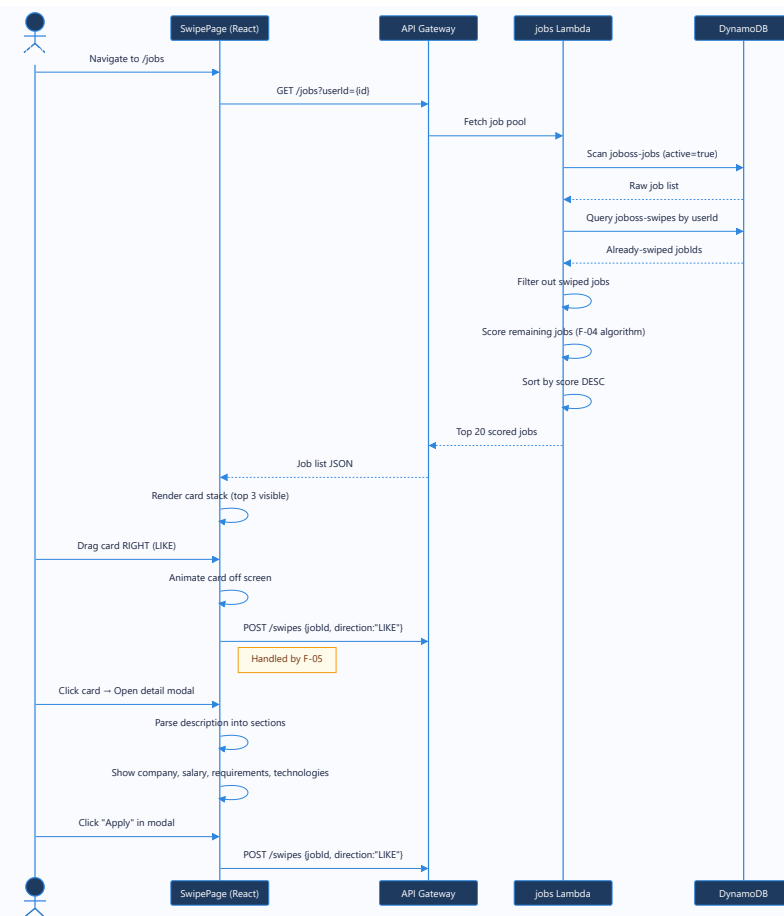


Fig F-03: Job Discovery & Swipe Interface

F-04

Job Filtering & Matching Algorithm

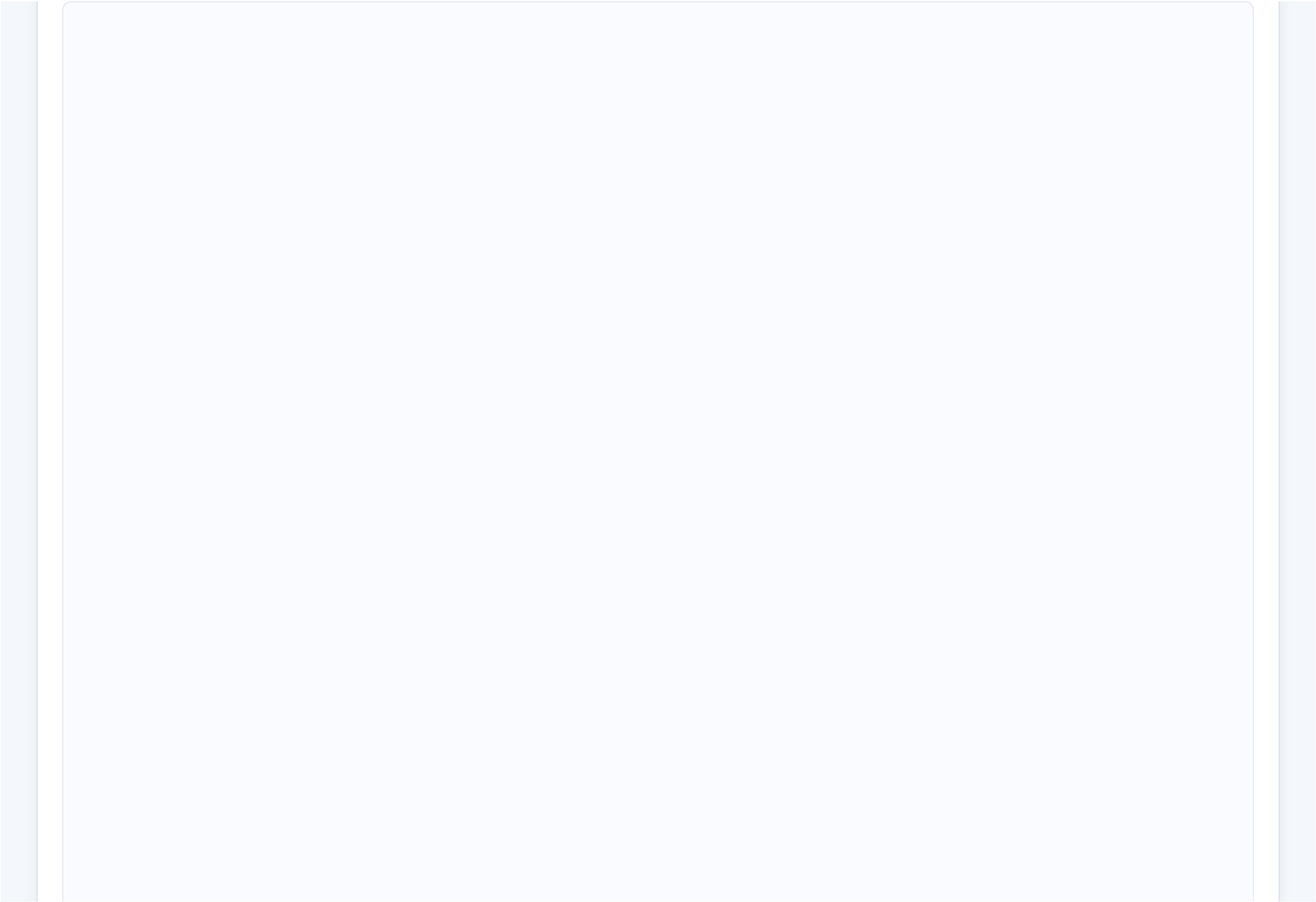
Actor: System (Internal)

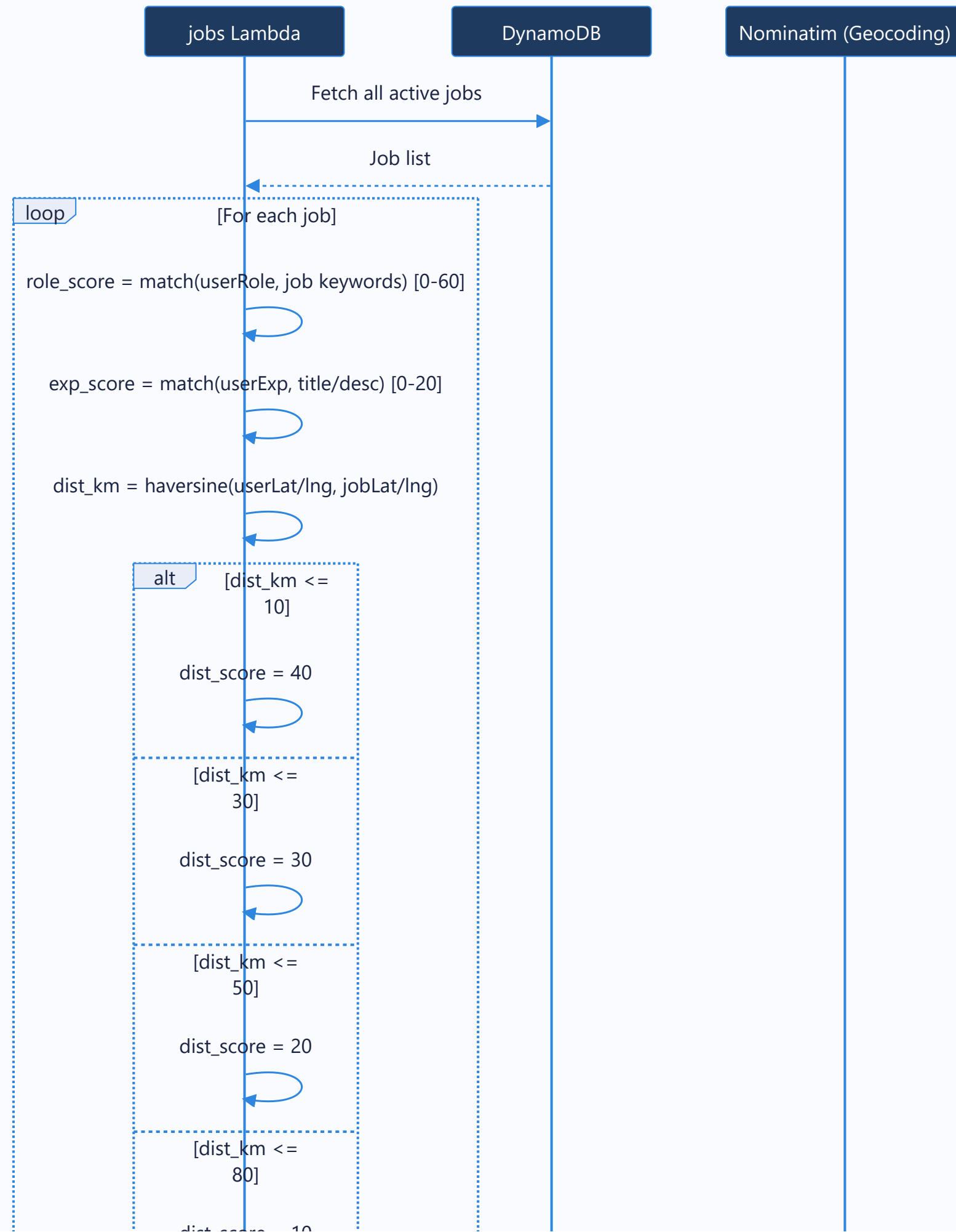
Scoring Engine

DynamoDB

Before serving jobs to the user, a multi-component scoring algorithm (0–100) ranks each job. Score components: **Role match** (keyword mapping across 15 categories), **Experience level** (Junior/Mid/Senior/Student detection), **Distance** (Haversine formula: 10 km = 40 pts, 80 km = 0 pts). High-scoring jobs form the "primary pool"; the rest form the "discovery pool".

Implements: `lambdas/jobs/handler.py` — `score_job()` , `haversine()` , `ROLE_KEYWORDS`





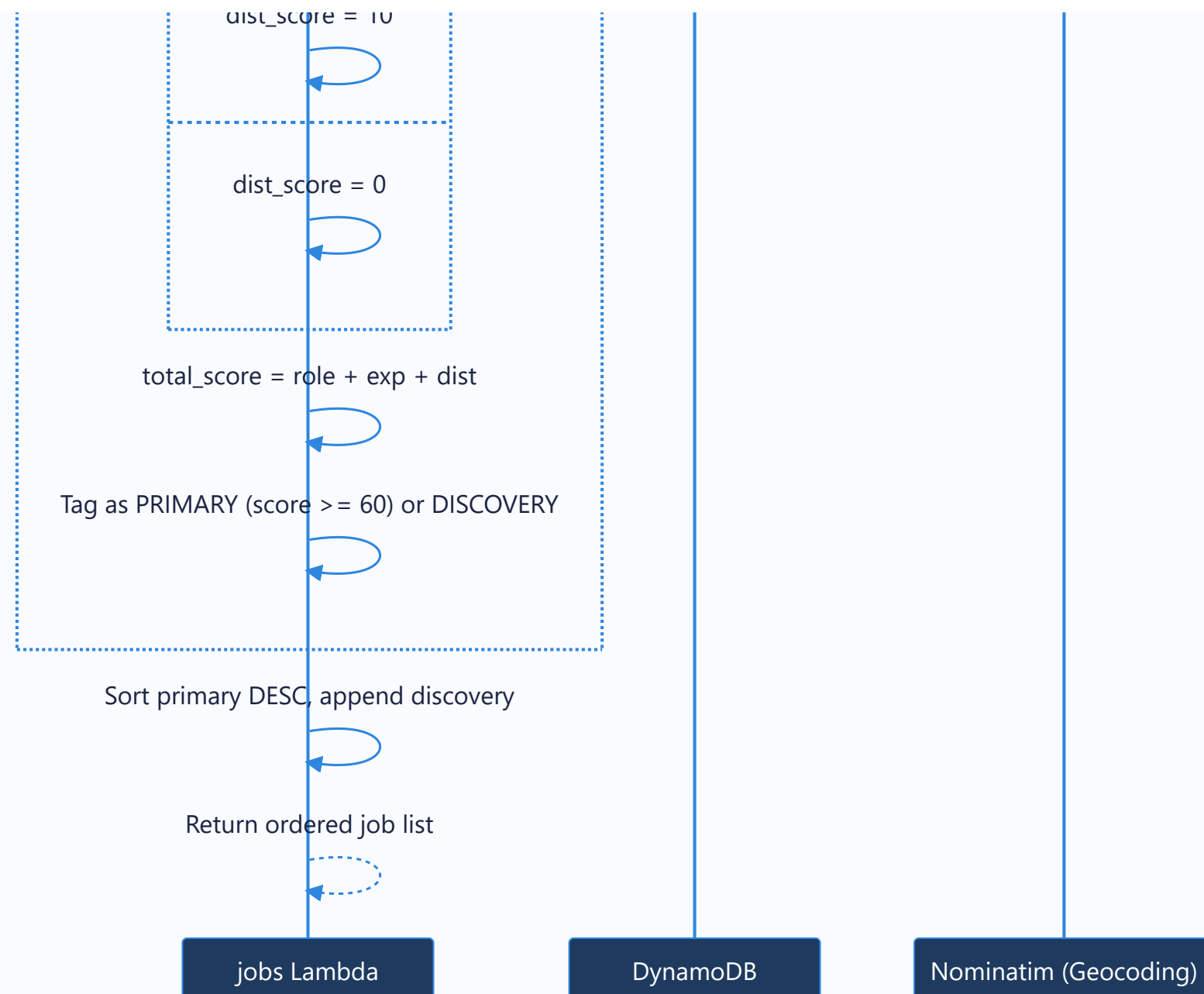


Fig F-04: Job Scoring & Matching Algorithm

F-05

Swipe Recording — Like & Pass

Actor: User

swipes Lambda

DynamoDB · SQS

All Tiers

When a user swipes RIGHT (LIKE), an *application* record is created with status pending. If Auto-Apply is enabled (PREMIUM+), a message is pushed to the SQS auto-apply queue. A PASS swipe records the decision without creating an application. A race-condition guard prevents duplicate swipes on the same job.

Implements: [lambdas/swipes/handler.py](#) [lambdas/applications/handler.py](#) [src/pages/SwipePage.jsx](#)

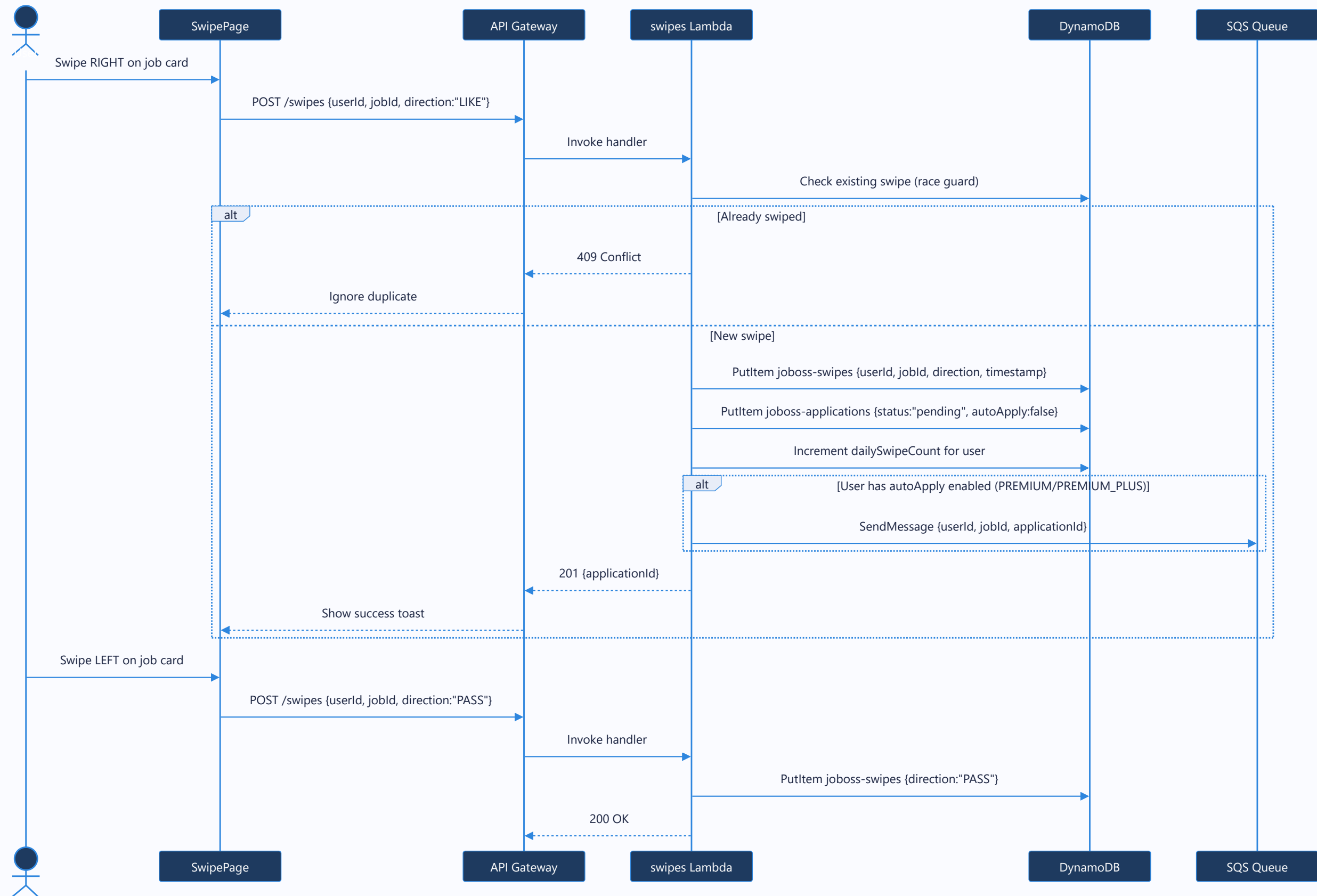


Fig F-05: Swipe Recording — Like & Pass

F-06

Undo Last Swipe

Actor: User

swipes Lambda

DynamoDB

All Tiers

Users can undo their most recent swipe. The swipe record and its associated application (if any) are deleted from DynamoDB, and the job card is re-inserted at the top of the visible stack. The daily swipe counter is decremented.

Implements: `lambdas/swipes/handler.py` — `DELETE /swipes/last` `src/pages/SwipePage.jsx` — Undo button

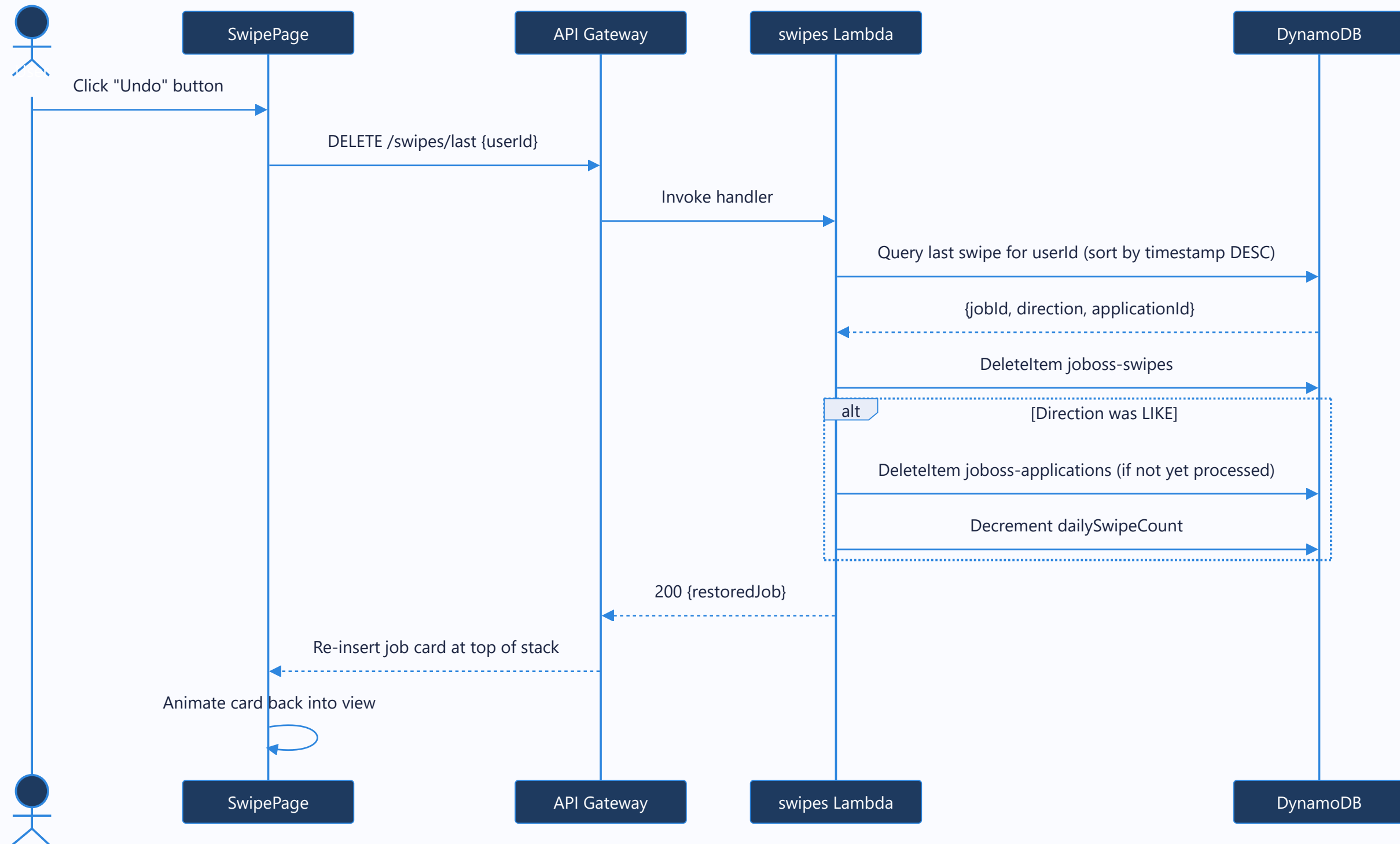


Fig F-06: Undo Last Swipe

F-07

Application Tracking (Dual-Track Status)

Actor: User

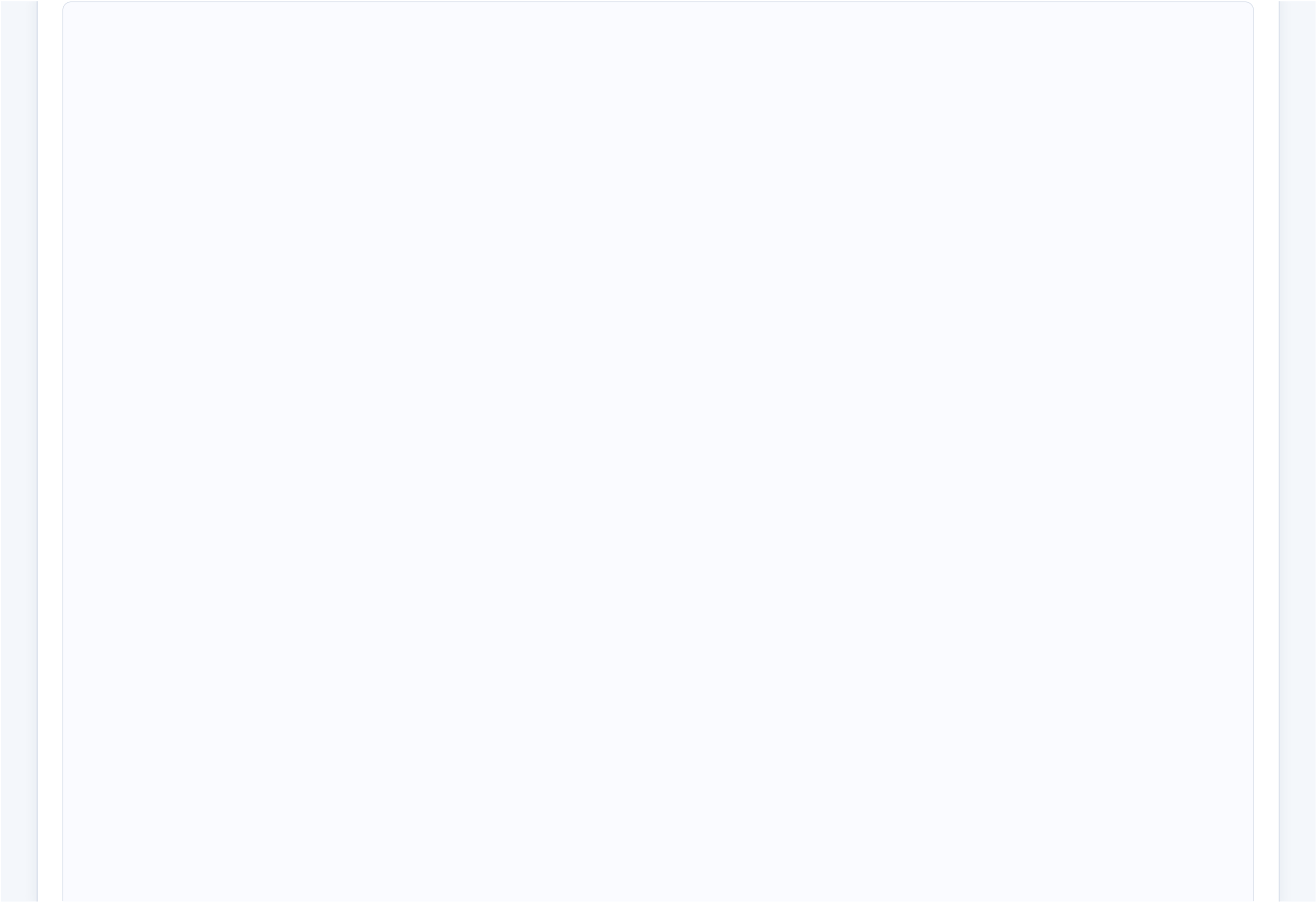
ApplicationsPage

DynamoDB

All Tiers

Applications have two independent status tracks. **Track A (User Funnel):** SUBMITTED → REVIEWED → INTERVIEW → ACCEPTED / REJECTED. **Track B (Auto-Apply System):** manual / pending / pending_tailoring / success / failed. Users can view, filter by tab, annotate with notes, and batch-delete applications.

Implements: `src/pages/ApplicationsPage.jsx` `lambdas/applications/handler.py`



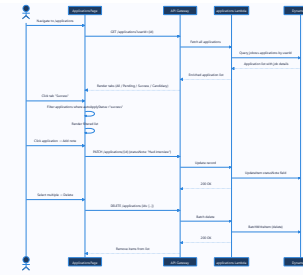


Fig F-07: Dual-Track Application Tracking



AI & Automation Features

F-08 → F-09

F-08

Auto-Apply Automation (ECS Fargate + Playwright)

Actor: System / User (PREMIUM+) ECS Fargate · Playwright SQS · SES · S3 PREMIUM / PREMIUM_PLUS

When a PREMIUM user swipes LIKE with Auto-Apply enabled, the system automatically navigates to the job's ATS platform using a headless Chromium browser (Playwright inside ECS Fargate), fills in the application form with the user's real details and resume, and submits. Supported platforms: SmartRecruiters, Lever, Greenhouse, Workday, Ashby, JobVite, Taleo, iCIMS, LinkedIn. The user receives an email notification for success or failure.

Implements: `lambdas/auto-apply/handler.py` `fargate/auto-apply/apply.py` `lambdas/swipes/handler.py` — SQS enqueue

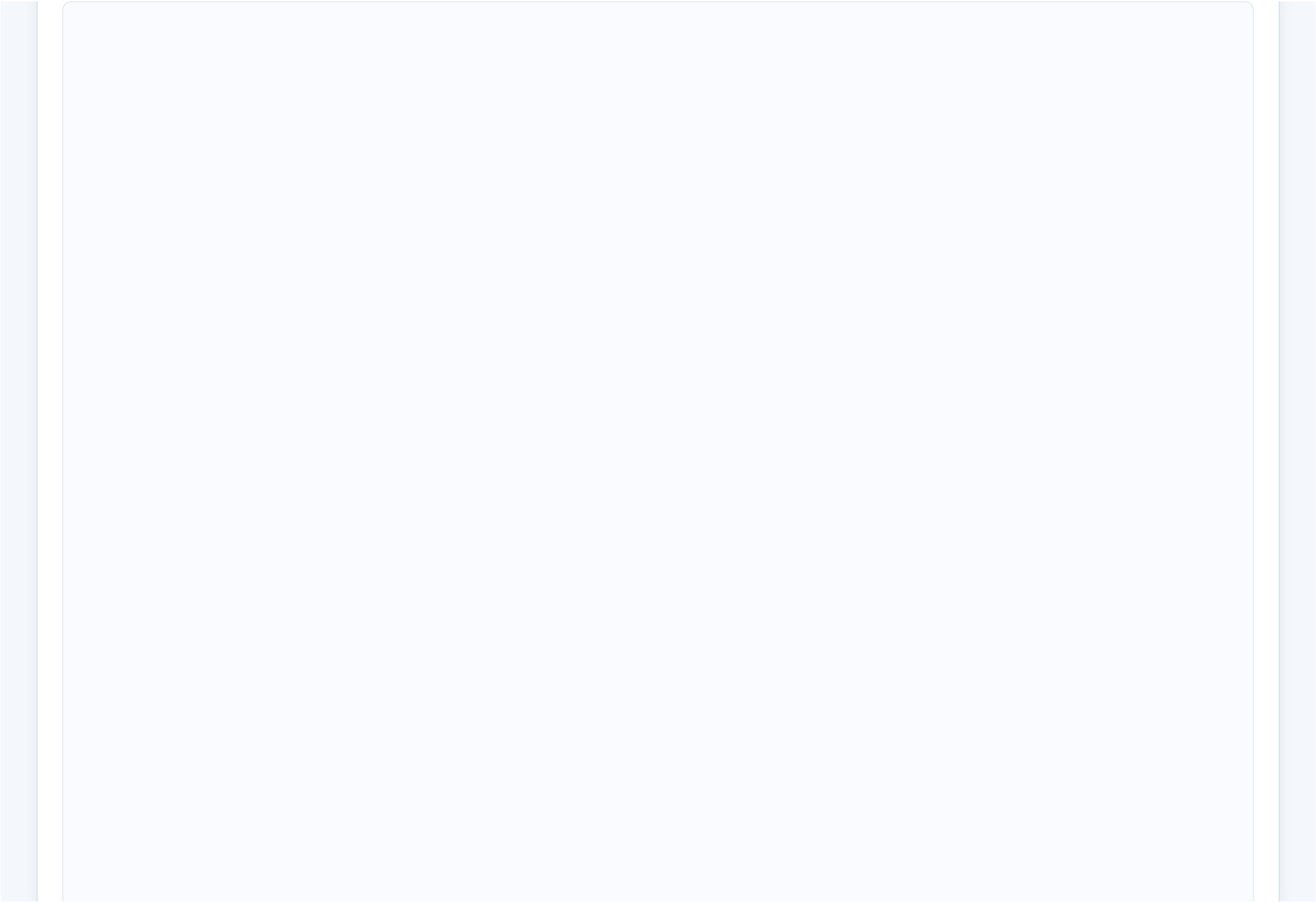




Fig F-08: Auto-Apply Automation Pipeline

F-09

AI Resume Tailoring (Claude via Bedrock)

Actor: User (PREMIUM+)

Amazon Bedrock

ai Lambda · S3

PREMIUM / PREMIUM_PLUS

PREMIUM users can request AI-tailored resumes for specific jobs. Claude Haiku (via Amazon Bedrock) rewrites the user's active resume to highlight relevant skills and keywords from the job description, producing an ATS-optimised PDF. PREMIUM tier is limited to 10 tailorings per month; PREMIUM_PLUS is unlimited.

Implements: `lambdas/ai/handler.py` `src/pages/ApplicationsPage.jsx` `src/pages/SwipePage.jsx`

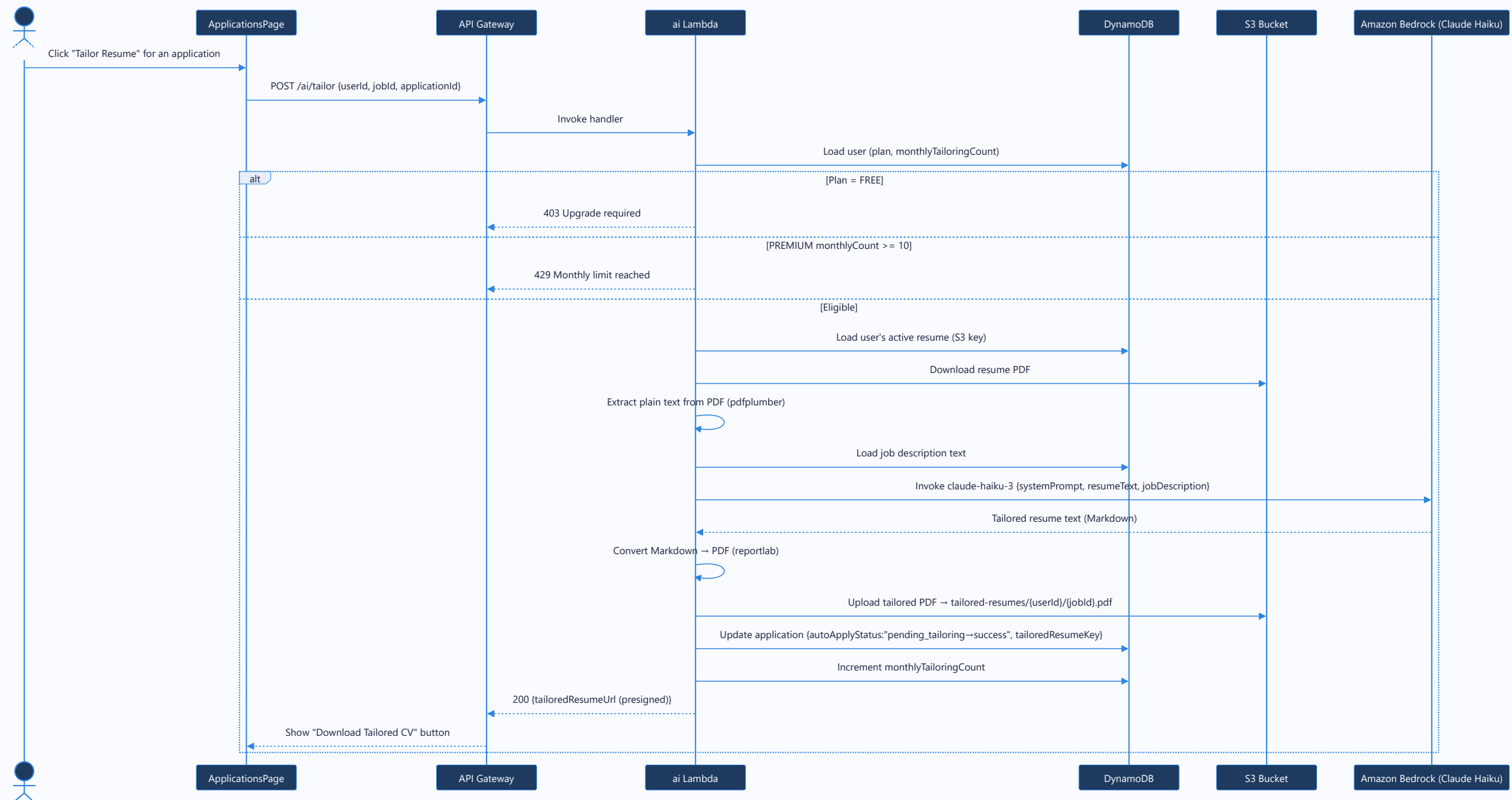
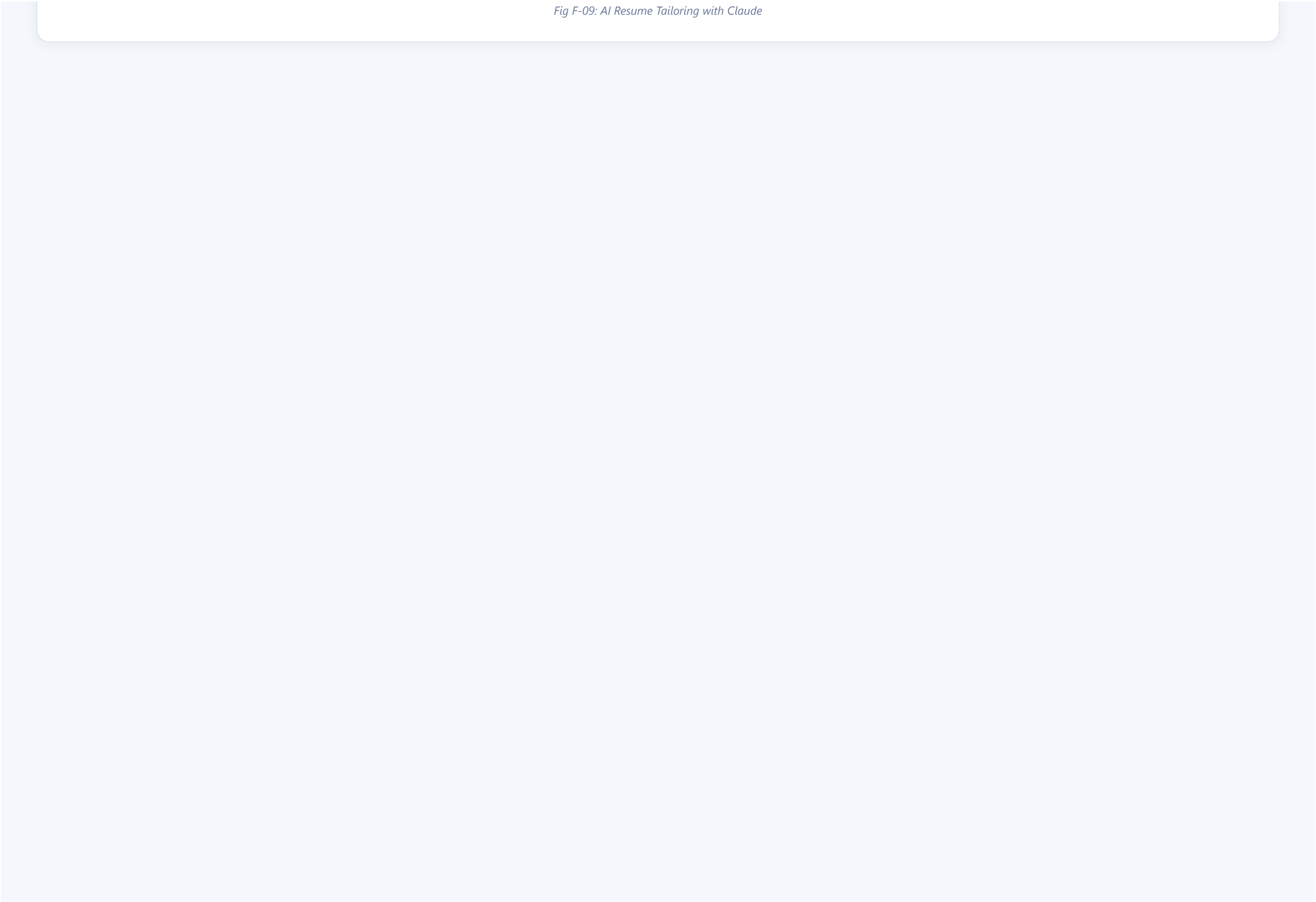


Fig F-09: AI Resume Tailoring with Claude





Profile & File Management

F-10 → F-12

F-10

Resume Upload & Management

Actor: User

uploads Lambda · S3

Presigned URLs

All Tiers

Users upload up to 3 resumes (PDF, max 5 MB). New uploads trigger a presigned S3 PUT URL; the file is streamed directly from the browser to S3 without passing through Lambda. If the user already has 3 resumes, the oldest is automatically deleted. Users can set any resume as the "active" resume used for applications.

Implements: `lambdas/uploads/handler.py` `lambdas/users/handler.py` `src/pages/ProfilePage.jsx`

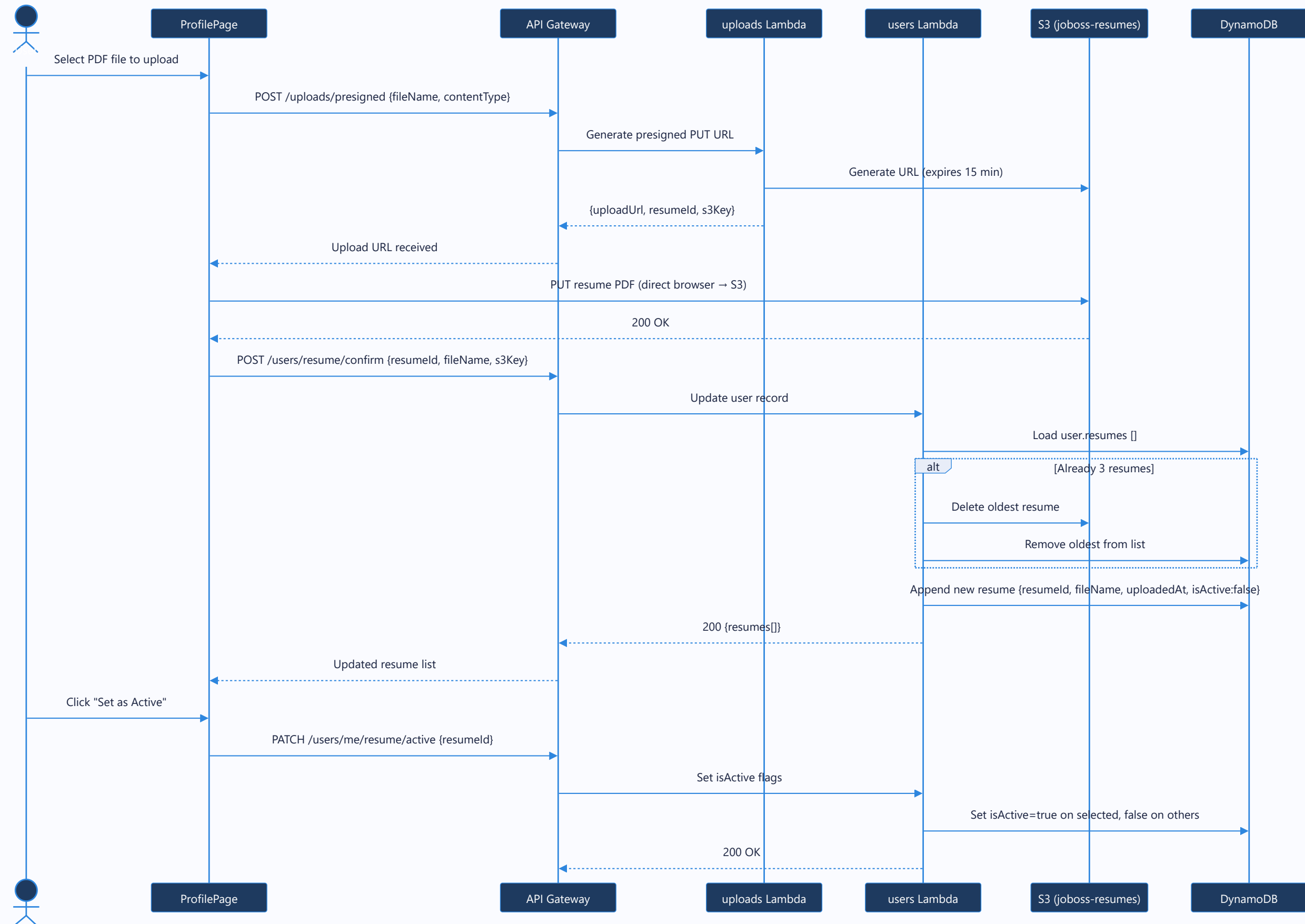


Fig F-10: Resume Upload & Management Flow

F-11

Profile Image Upload & Removal

Actor: User

profile-image Lambda

S3 · Pillow

All Tiers

Users upload a profile avatar (JPG, PNG, WebP, GIF — max 5 MB). The Lambda downscales the image to a maximum of 512 px using Pillow and saves it as a JPEG at 85% quality to S3. The public URL is stored in the user record. Users can also remove their profile image.

Implements: `lambdas/profile-image/handler.py` `src/pages/ProfilePage.jsx`

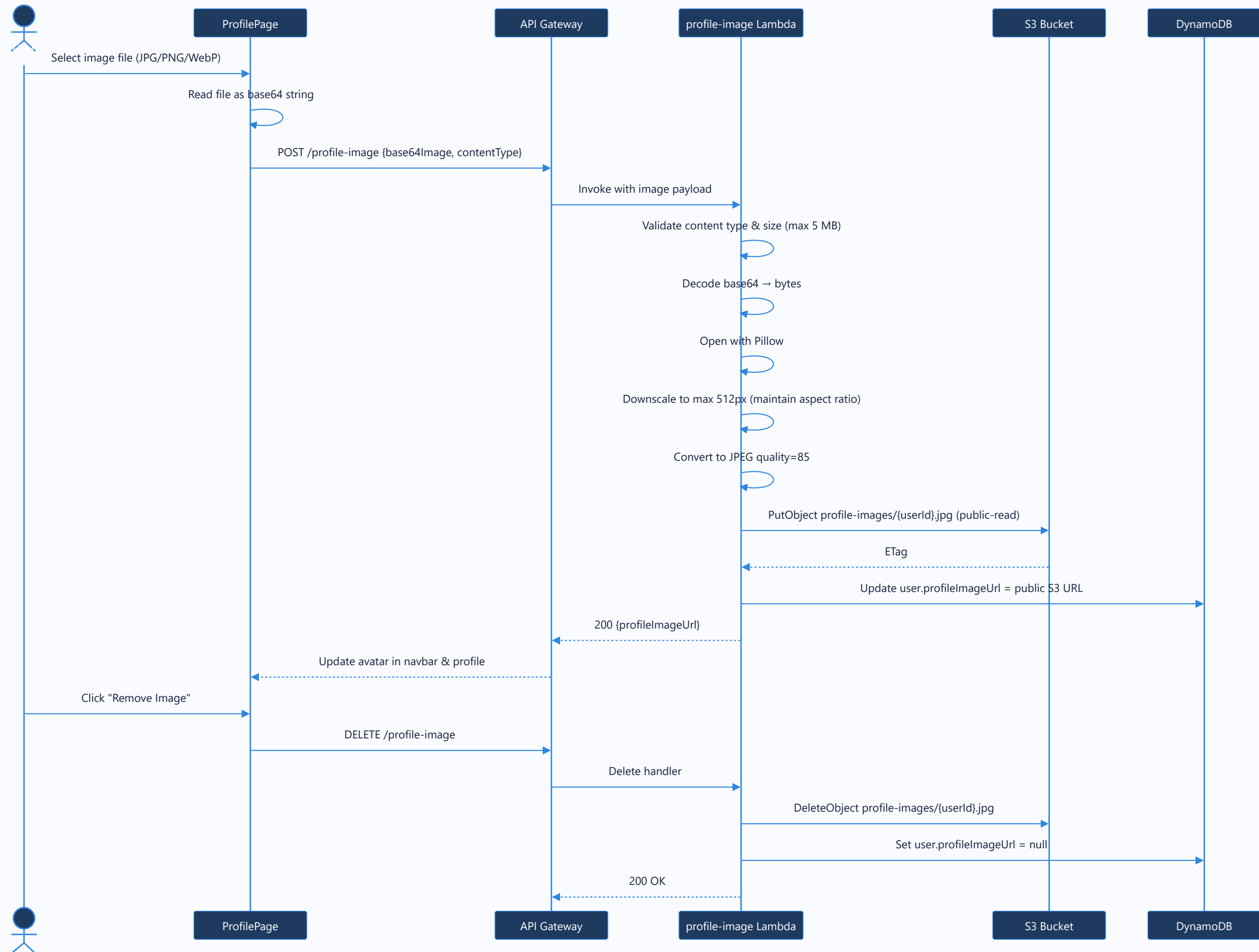


Fig F-11: Profile Image Upload & Removal

F-12

Subscription & Stripe Payment

Actor: User

subscriptions Lambda

Stripe · DynamoDB

PREMIUM

PREMIUM_PLUS

Three subscription tiers: FREE (default), PREMIUM, PREMIUM_PLUS. Payment is processed by Stripe Checkout with a 7-day free trial. Stripe webhooks update the DynamoDB subscription record on events: `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`. Plan changes are propagated to the user's swipe quota and feature entitlements in real time.

Implements: `lambdas/subscriptions/handler.py` `src/pages/SubscriptionPage.jsx` `src/pages/ProfilePage.jsx`

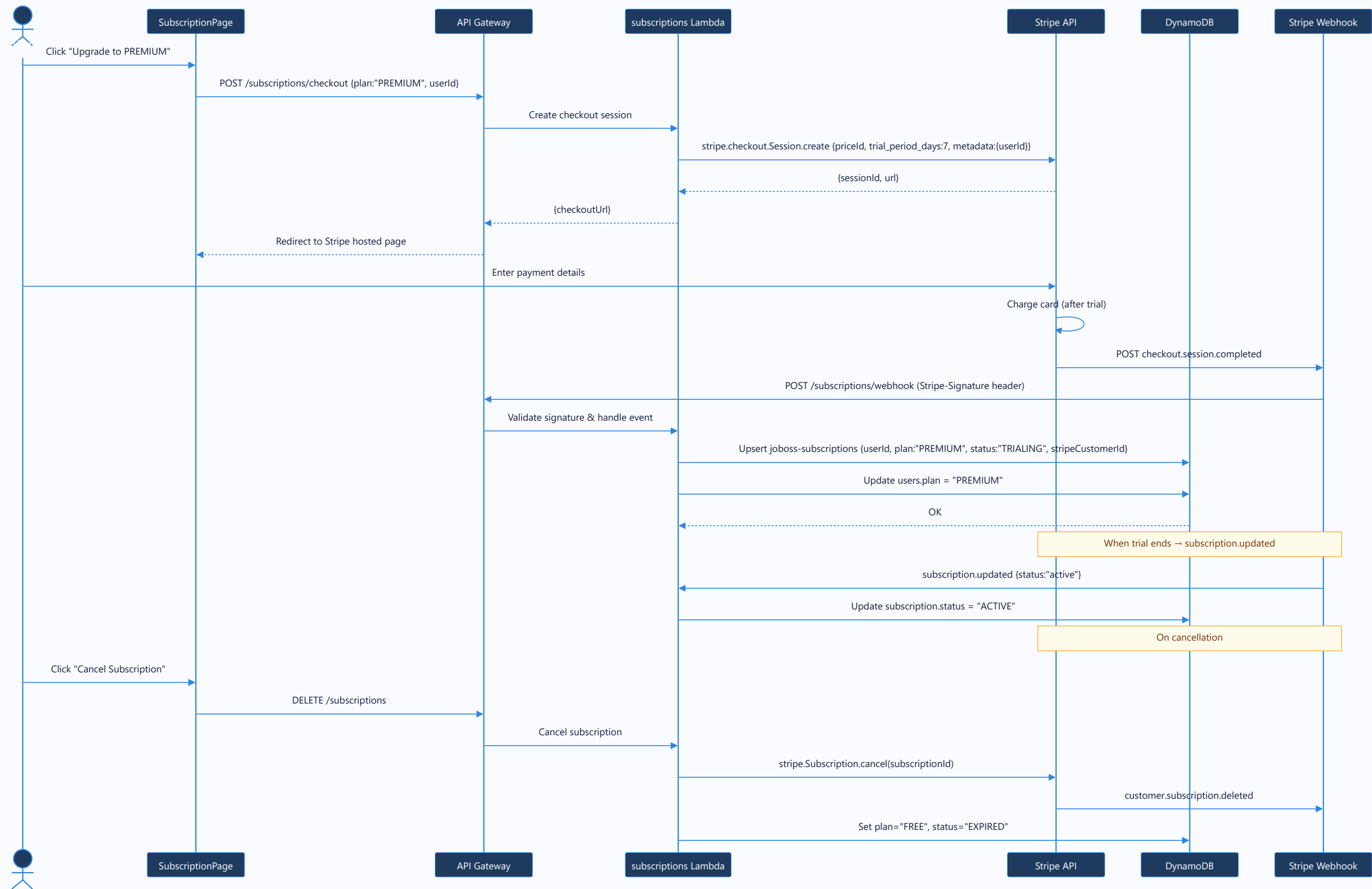


Fig F-12: Stripe Subscription Lifecycle



Quota & Entitlement Enforcement

F-13

F-13

Daily Swipe Quota Enforcement

Actor: User / System swipes Lambda DynamoDB FREE: 5 | PREMIUM: 30 | PREMIUM_PLUS: ∞

Each tier has a daily swipe limit enforced at the Lambda level. FREE = 5, PREMIUM = 30, PREMIUM_PLUS = unlimited. The counter resets daily at midnight UTC. When a quota is reached, the frontend shows a `LimitModal` with an upgrade prompt. Failed auto-applies are refunded (`quotaExempt` flag) so users are not penalised.

Implements: `lambdas/swipes/handler.py` — quota check `src/components/LimitModal.jsx` `src/pages/SwipePage.jsx`

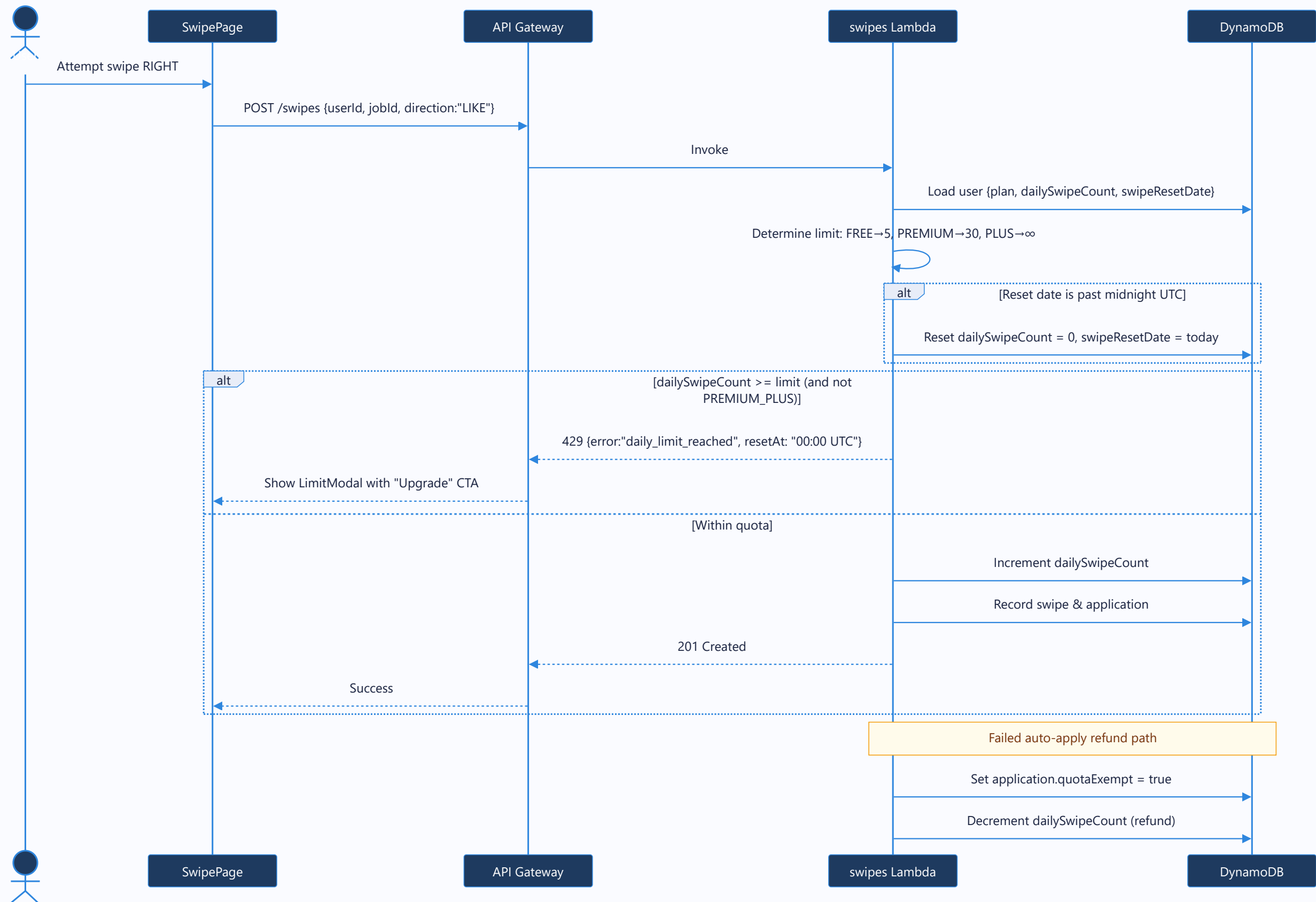


Fig F-13: Daily Swipe Quota Enforcement



Administration

F-14 → F-16

F-14

Admin Dashboard & Statistics

Actor: Admin

admin Lambda · AdminPage

Cognito ADMIN group

Admins (Cognito ADMIN group members) access a protected dashboard showing platform-wide statistics: total users, total jobs, total applications, user distribution by plan, and application status breakdown. The dashboard also provides entry points to user management and job management modules.

Implements: `src/pages/AdminPage.jsx` `lambdas/admin/handler.py`

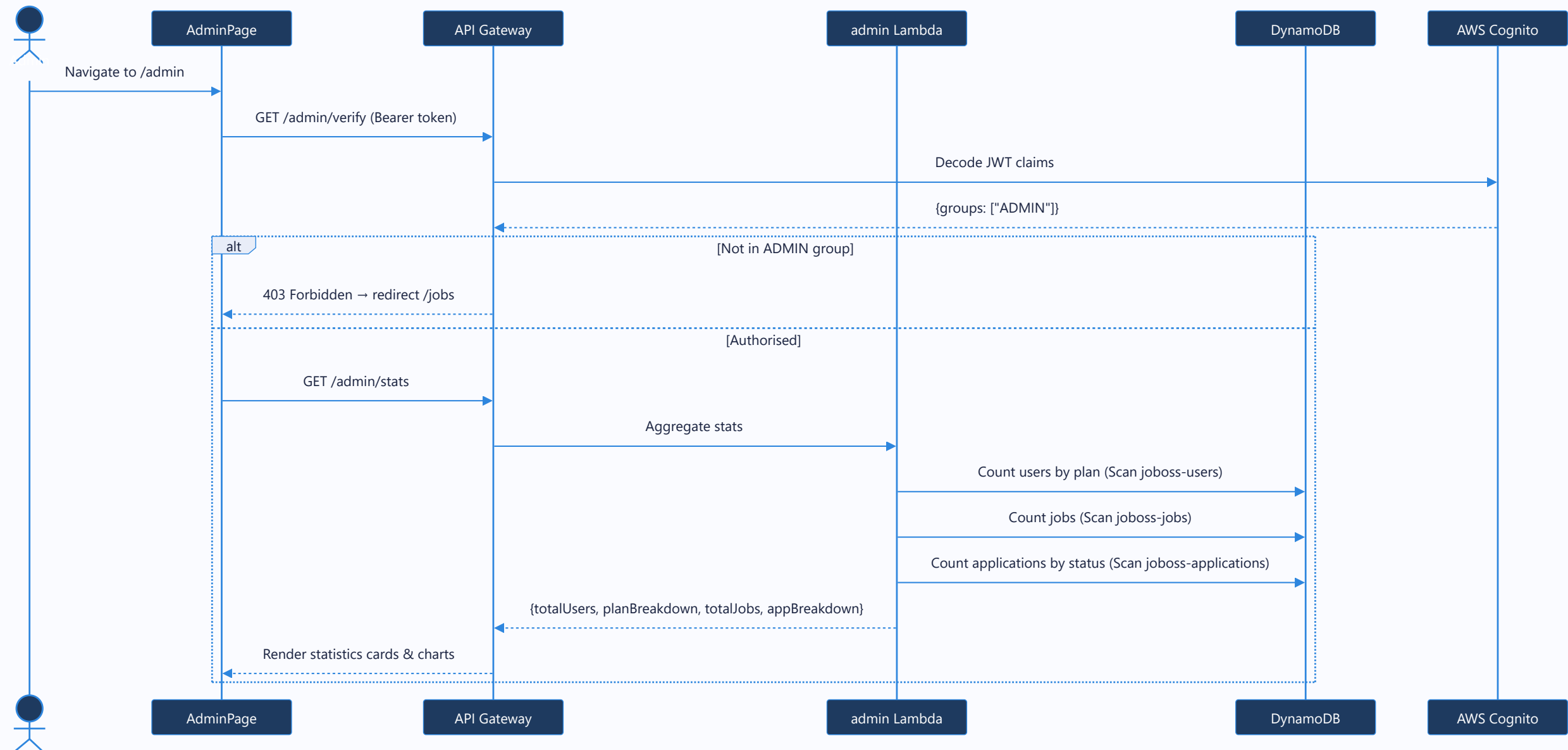


Fig F-14: Admin Dashboard Statistics

F-15

Admin User Management

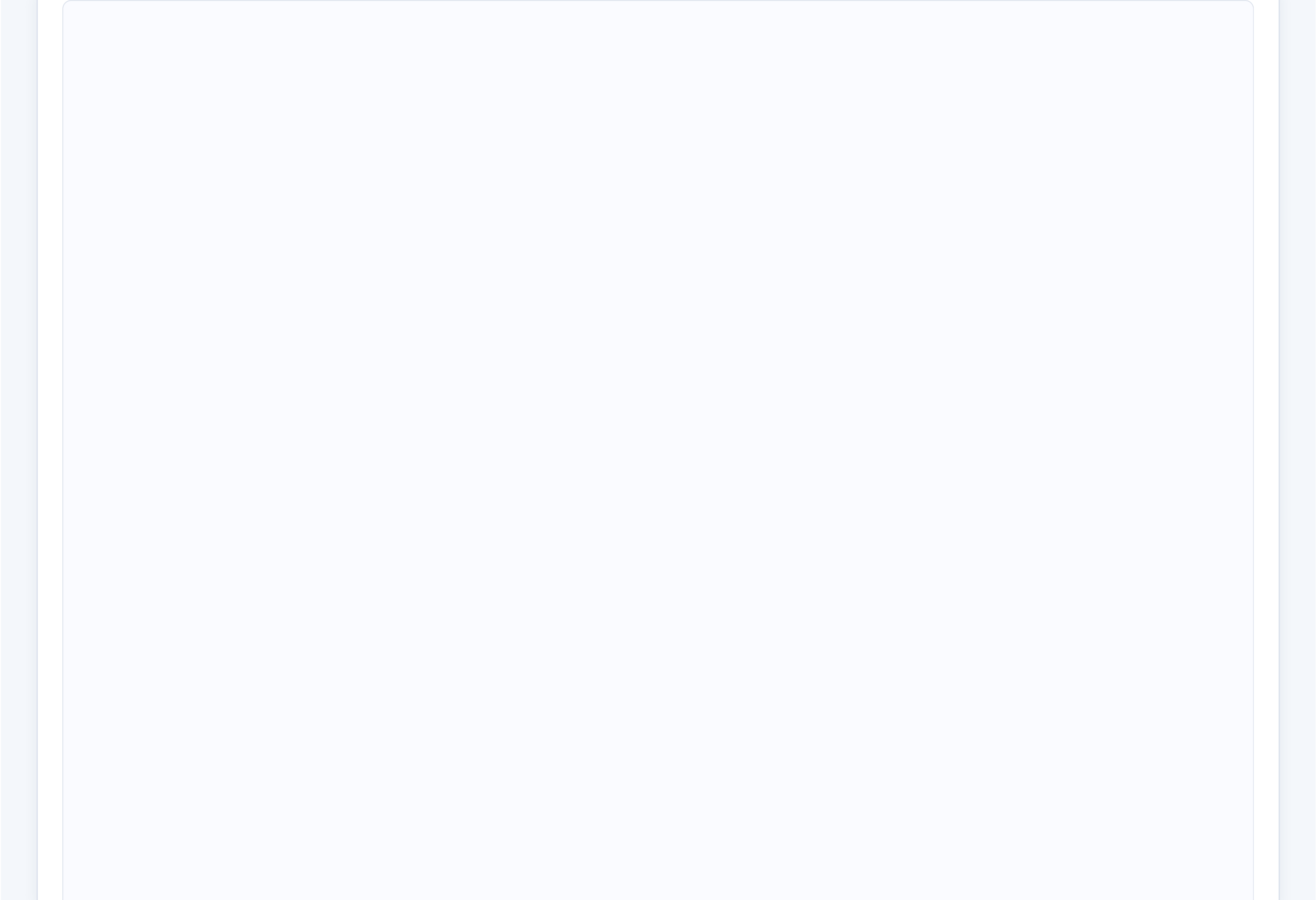
Actor: Admin

admin Lambda

Cognito · DynamoDB

Admins can view all registered users, change their subscription plan, reset daily swipe quotas, block/unblock accounts, delete users with all associated data, and grant or revoke admin privileges (requires password re-verification to prevent privilege escalation).

Implements: `lambdas/admin/handler.py` `src/pages/AdminPage.jsx`



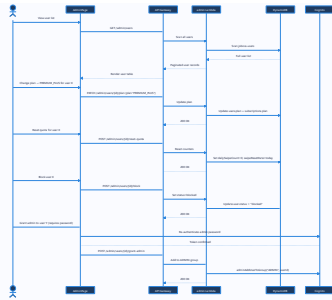


Fig F-15: Admin User Management Actions

F-16

Admin Job Management

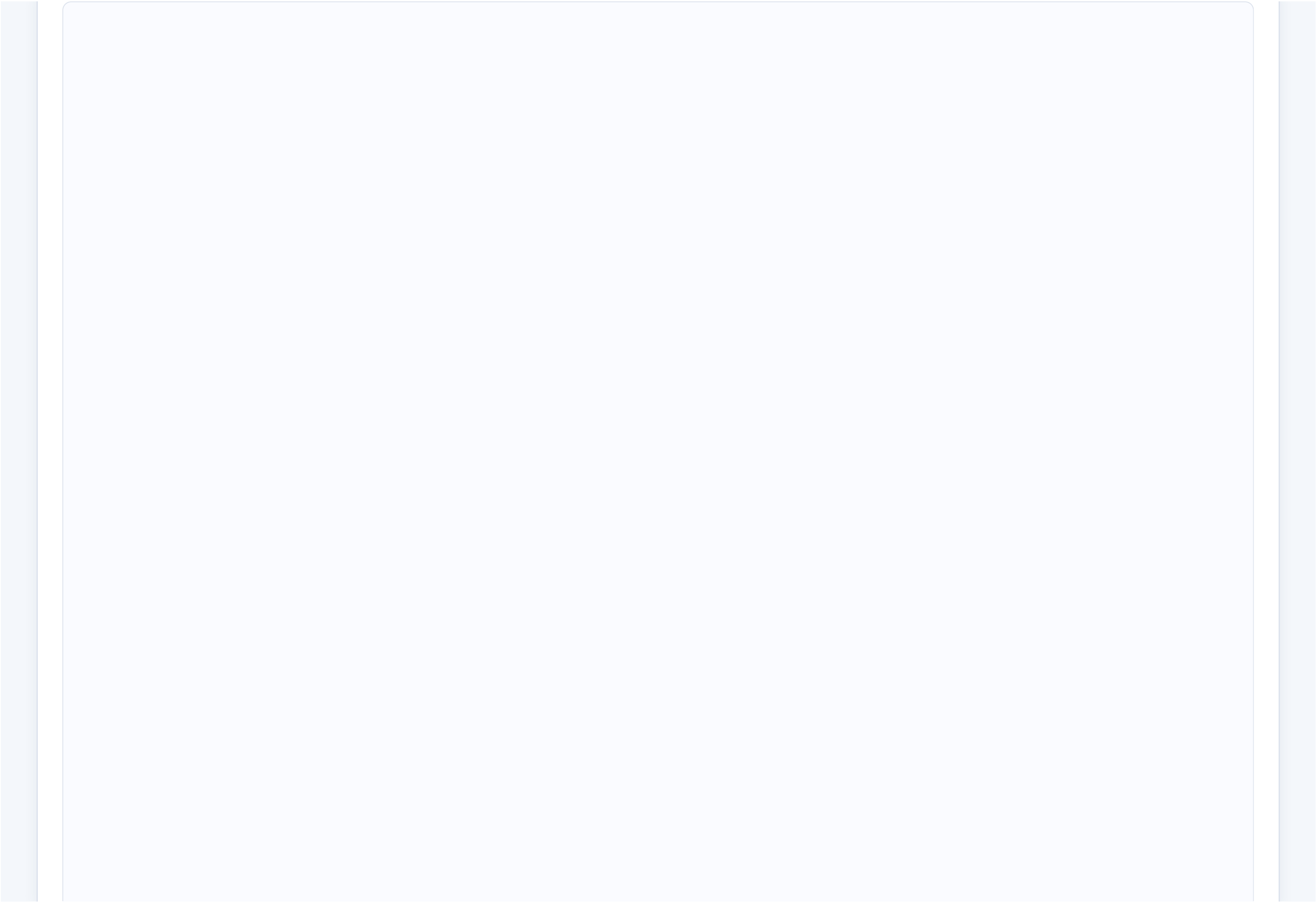
Actor: Admin

admin Lambda · jobs Lambda

DynamoDB

Admins can view the full list of jobs in the system, toggle individual jobs active/inactive (controlling visibility in the swipe feed), and manually trigger the job importer Lambda to pull new postings from the Telegram channel.

Implements: `lambdas/admin/handler.py` `lambdas/jobs_importer/handler.py` `src/pages/AdminPage.jsx`



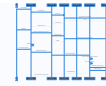


Fig F-16: Admin Job Management & Importer Trigger



Data Ingestion & Maintenance

F-17 → F-18

F-17

Job Importer (Telegram Channel Ingestion)

Actor: System (Scheduled) jobs_importer Lambda Bedrock · DynamoDB · EventBridge

A scheduled Lambda (via EventBridge) uses the Telethon library to read new job postings from a curated Telegram channel. Each post is parsed for title, description, company, location, and apply URL. The job description is normalised via Bedrock AI. Coordinates are resolved via Nominatim geocoding. Duplicate jobs (by source_job_id) are skipped.

Implements: `lambdas/jobs_importer/handler.py` `lambdas/job_description_ai/handler.py`

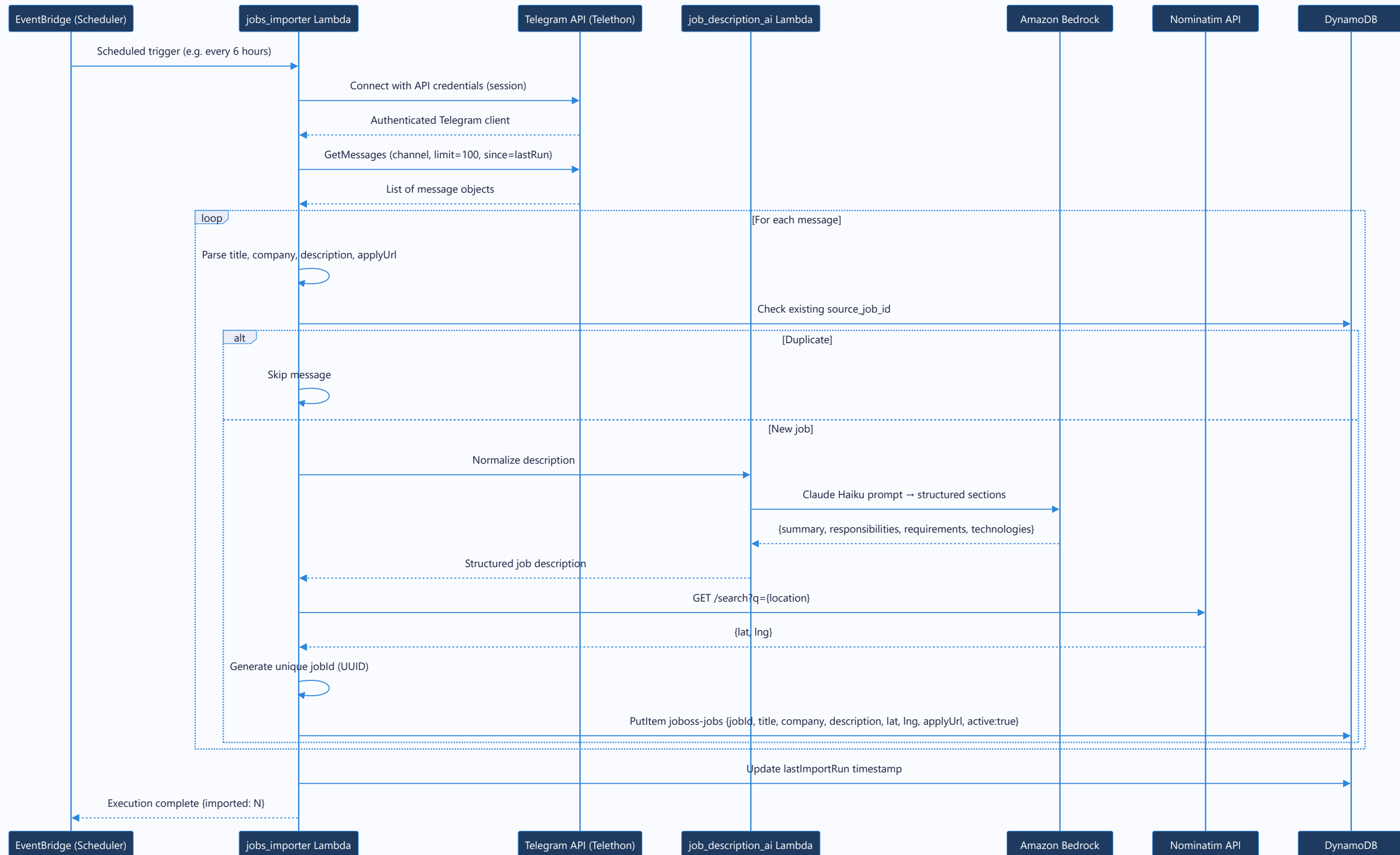


Fig F-17: Telegram Job Ingestion Pipeline

F-18

Job Status Monitoring & Cleanup

Actor: System (Scheduled)

jobs_status_checker Lambda

EventBridge · DynamoDB

A daily scheduled Lambda checks all active job postings for staleness. Jobs whose apply URL returns a non-200 HTTP response, or that are older than a configured TTL (e.g. 30 days), are automatically marked inactive and removed from the swipe feed.

Implements: `lambdas/jobs_status_checker/handler.py`

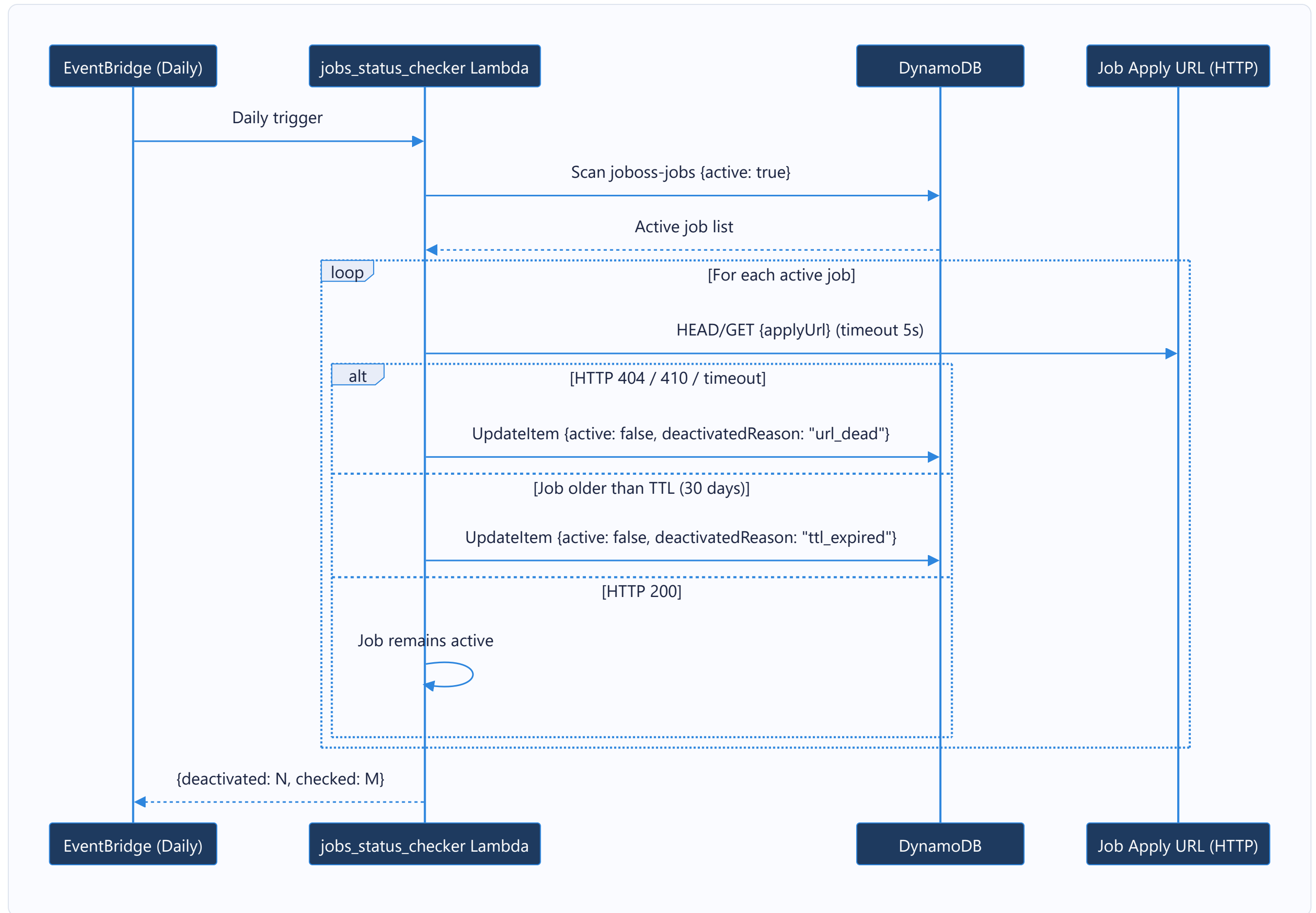


Fig F-18: Automated Job Status Monitoring



Integrations & Notifications

F-19 → F-20

F-19

Chrome Extension — ATS Auto-Fill

Actor: User

Chrome Extension

S3 Presigned URLs

All Tiers

A companion Chrome extension allows users to auto-fill job application forms on any ATS platform directly in the browser. The extension authenticates via the main JoBoss OAuth flow, fetches the user profile from the backend API, detects the current ATS platform, and injects the user's details and resume into form fields.

Implements: `chrome-extension/popup.html` `chrome-extension/content.js` `chrome-extension/background.js` `src/pages/AuthExtensionPage.jsx`

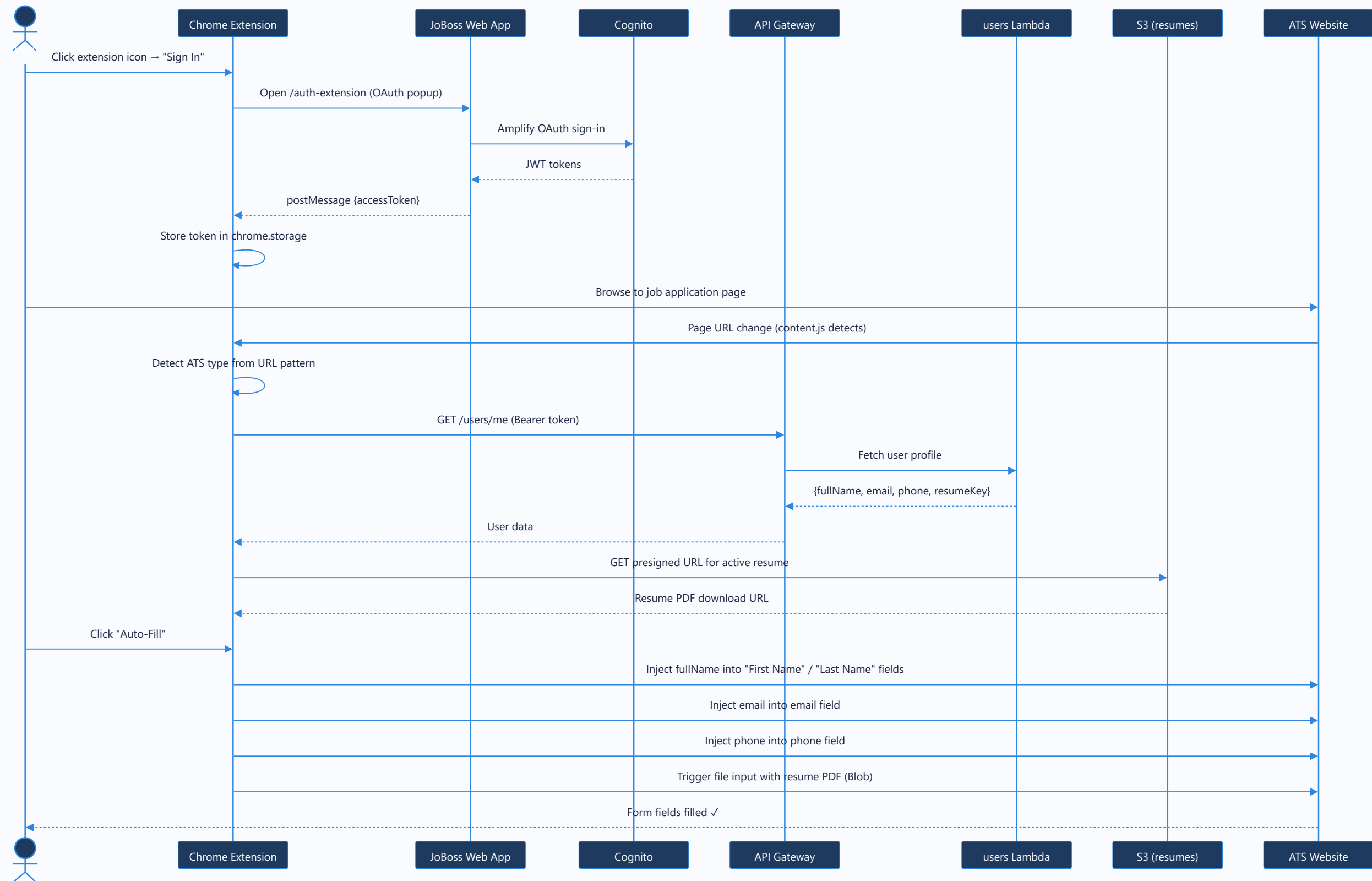


Fig F-19: Chrome Extension Auto-Fill Flow

F-20

Email Notification System (Amazon SES)

Actor: System → User

Amazon SES

auto-apply Lambda

PREMIUM / PREMIUM_PLUS

After every Auto-Apply attempt, the system sends a transactional email to the user via Amazon SES. A success email includes the job title, company name, and a confirmation message. A failure email includes the reason the application could not be submitted and encourages the user to apply manually.

Implements: `lambdas/auto-apply/handler.py` — `send_ses_email()`



Syntax error in text
mermaid version 10.9.6

Fig F-20: SES Email Notification Flow



Personal Dashboard & Profile Settings

F-21 · F-23 · F-28 · F-29

F-21

Personal Dashboard — At-a-Glance Statistics

Actor: User

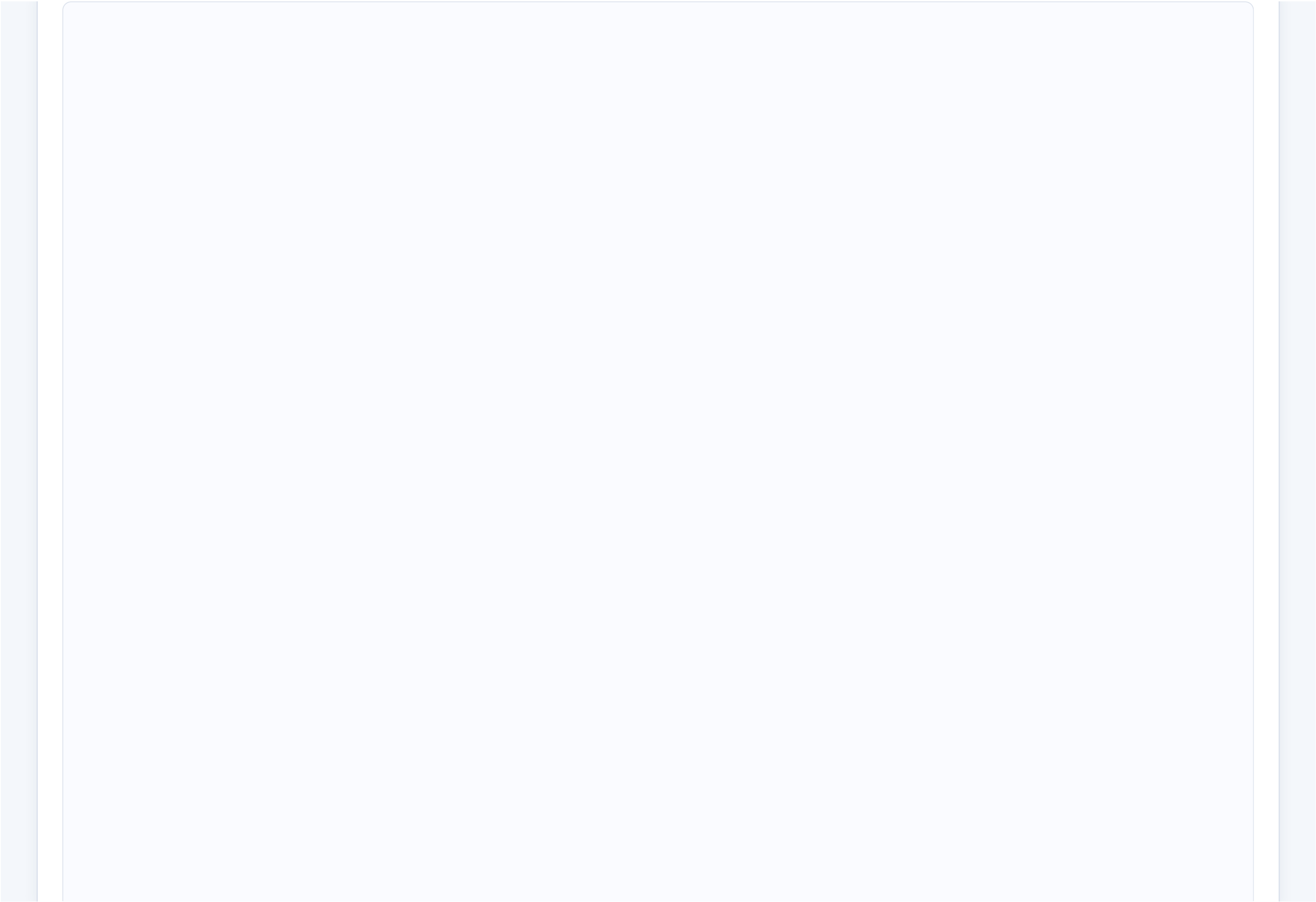
DashboardPage

DynamoDB

All Tiers

A dedicated /dashboard page gives users a personal snapshot of their job-search progress: total applications submitted, how many were accepted / rejected / pending, the number of daily swipes remaining (with plan context), and a scrollable application history list with status badges. A prominent upgrade CTA is shown to FREE users.

Implements: `frontend/src/pages/DashboardPage.jsx` `lambdas/applications/handler.py` `lambdas/users/handler.py`



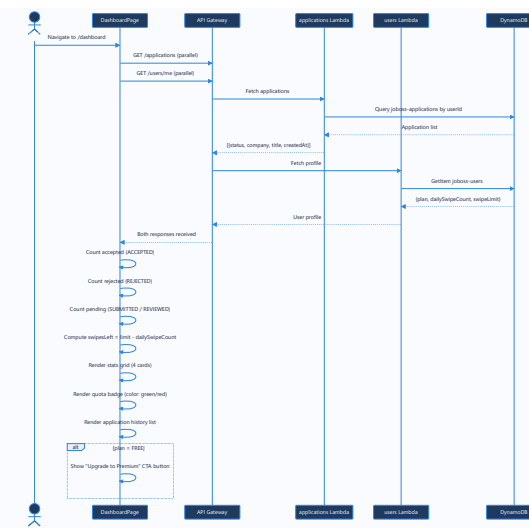


Fig F-21: Personal Dashboard Statistics Page

F-23

Edit Profile & Change Password

Actor: User

ProfileModals

Cognito · DynamoDB

All Tiers

From a dropdown anchored to the profile avatar, users access two bottom-sheet modals: **Edit Profile** — update full name and email stored in DynamoDB; **Change Password** — supply current and new password, validated against Cognito via AWS Amplify `updatePassword()`. On save, a `profile-updated` custom event is broadcast so the Navbar and all components refresh instantly without a page reload.

Implements: `frontend/src/components/ProfileModals.jsx` `frontend/src/pages/ProfilePage.jsx` `lambdas/users/handler.py`

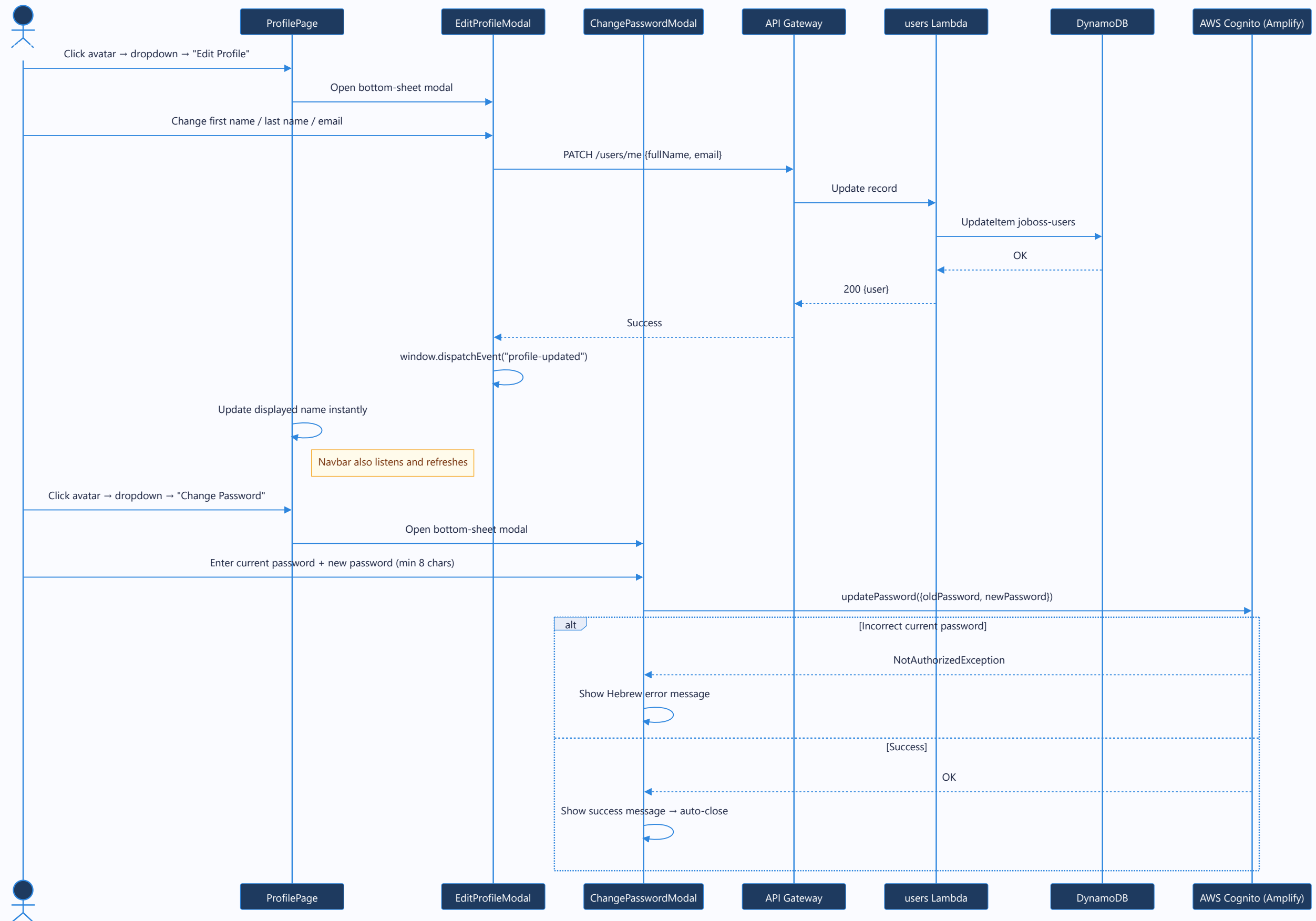


Fig F-23: Edit Profile & Change Password Modals

F-28

Show All Jobs Toggle (Permanent Discovery Mode)

Actor: User

ProfilePage · SwipePage

DynamoDB · localStorage

All Tiers

A toggle in the Profile settings card lets users permanently disable preference-based filtering and always receive all available jobs, regardless of role or location match. The setting is persisted both in `localStorage` (immediate effect) and in the DynamoDB user record (cross-device sync). When active, it overrides the temporary 30-minute Discovery Mode (F-26).

Implements: `frontend/src/pages/ProfilePage.jsx` — `showAllJobs` state `frontend/src/pages/SwipePage.jsx` — `showAllJobsPref` / `discoveryActive` `lambdas/users/handler.py`

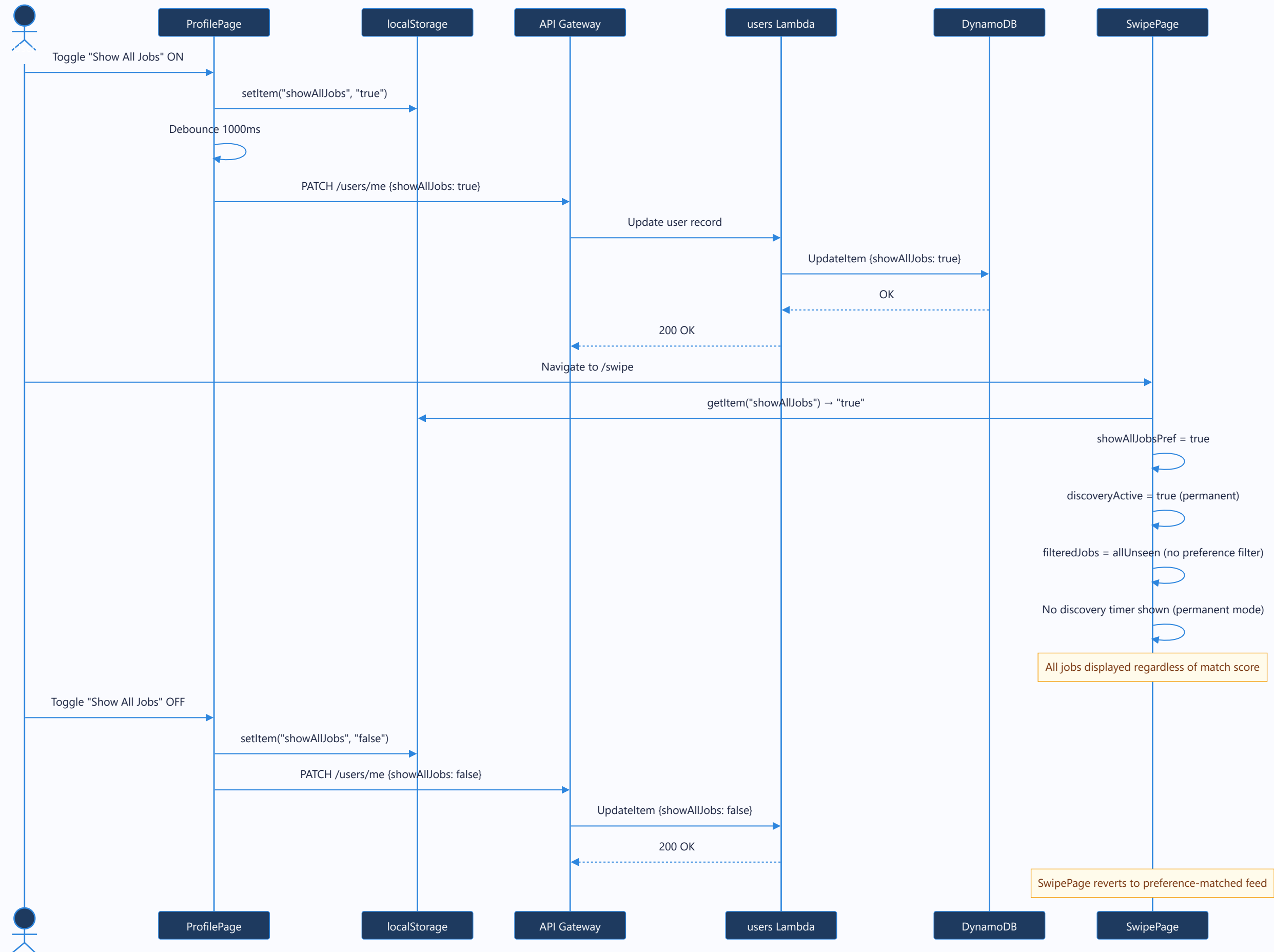


Fig F-28: Show All Jobs (Permanent Discovery) Toggle

F-29

GPS Location Auto-Detection

Actor: User

ProfilePage · Browser Geolocation API

Nominatim (Reverse Geocoding)

All Tiers

In the Location Preferences section of the Profile page, users can tap "Detect Current Location Automatically." The browser's Geolocation API is invoked to obtain GPS coordinates, which are then reverse-geocoded via Nominatim to a human-readable city name. The coordinates are stored in `localStorage` and the user record so the matching algorithm (F-04) can compute distances.

Implements: `frontend/src/pages/ProfilePage.jsx` — `handleUseCurrentLocation()` `frontend/src/components/LocationInput.jsx` `lambdas/users/handler.py`

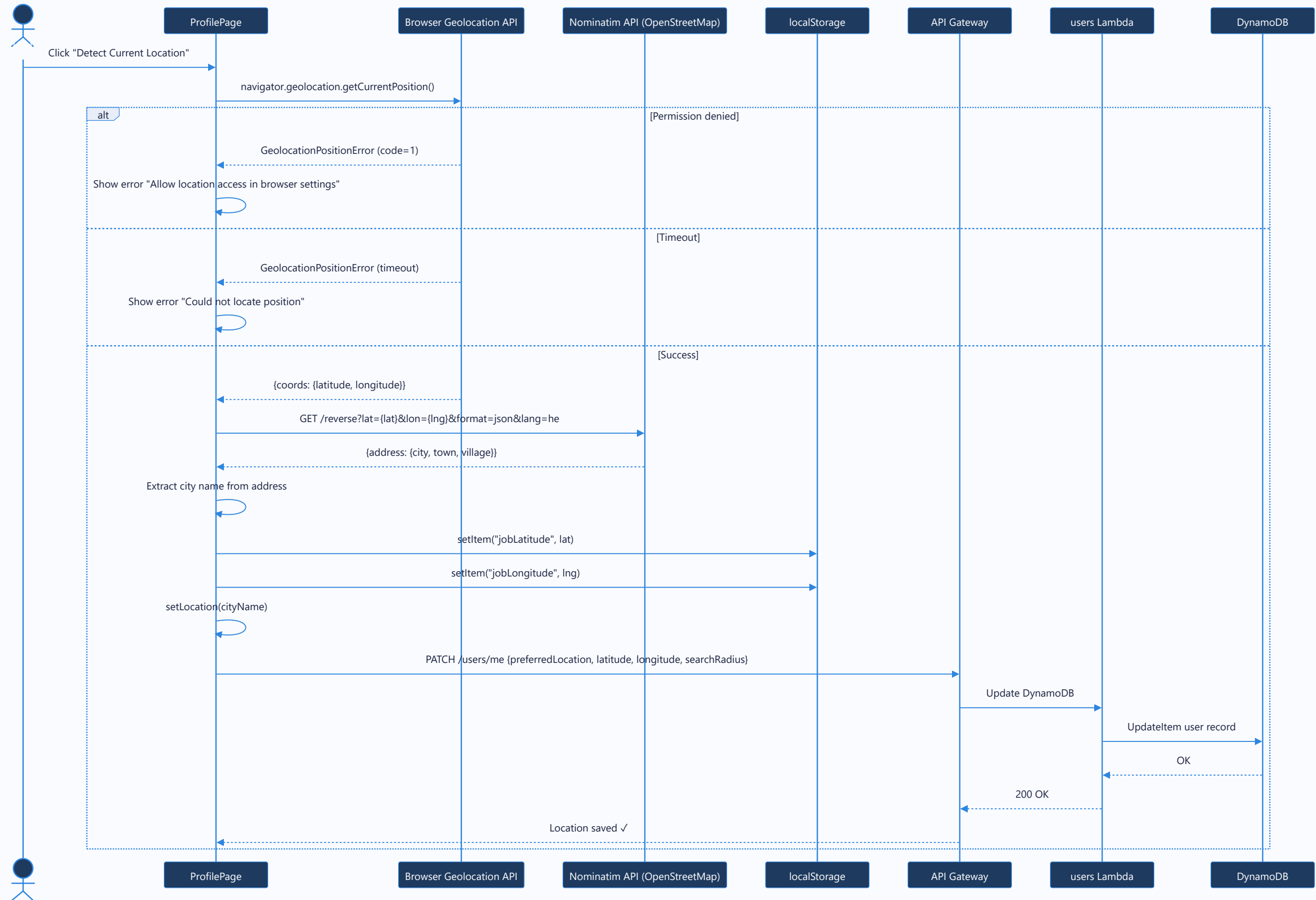


Fig F-29: GPS Location Auto-Detection Flow



Authentication & Access Control — Extended

F-24 · F-25

F-24

Account Suspension Screen

Actor: Suspended User App.jsx · SuspendedScreen Cognito · DynamoDB All Tiers

When the backend returns a 403 ACCOUNT_SUSPENDED response during the app startup auth check, the entire UI is replaced with a dedicated suspension screen. It displays the JoBoss logo, a Hebrew message explaining the account is blocked, a support email link, and a sign-out button. The user cannot navigate to any other page while suspended.

Implements: frontend/src/App.jsx — SuspendedScreen component, isSuspended state lambdas/users/handler.py — ACCOUNT_SUSPENDED error code

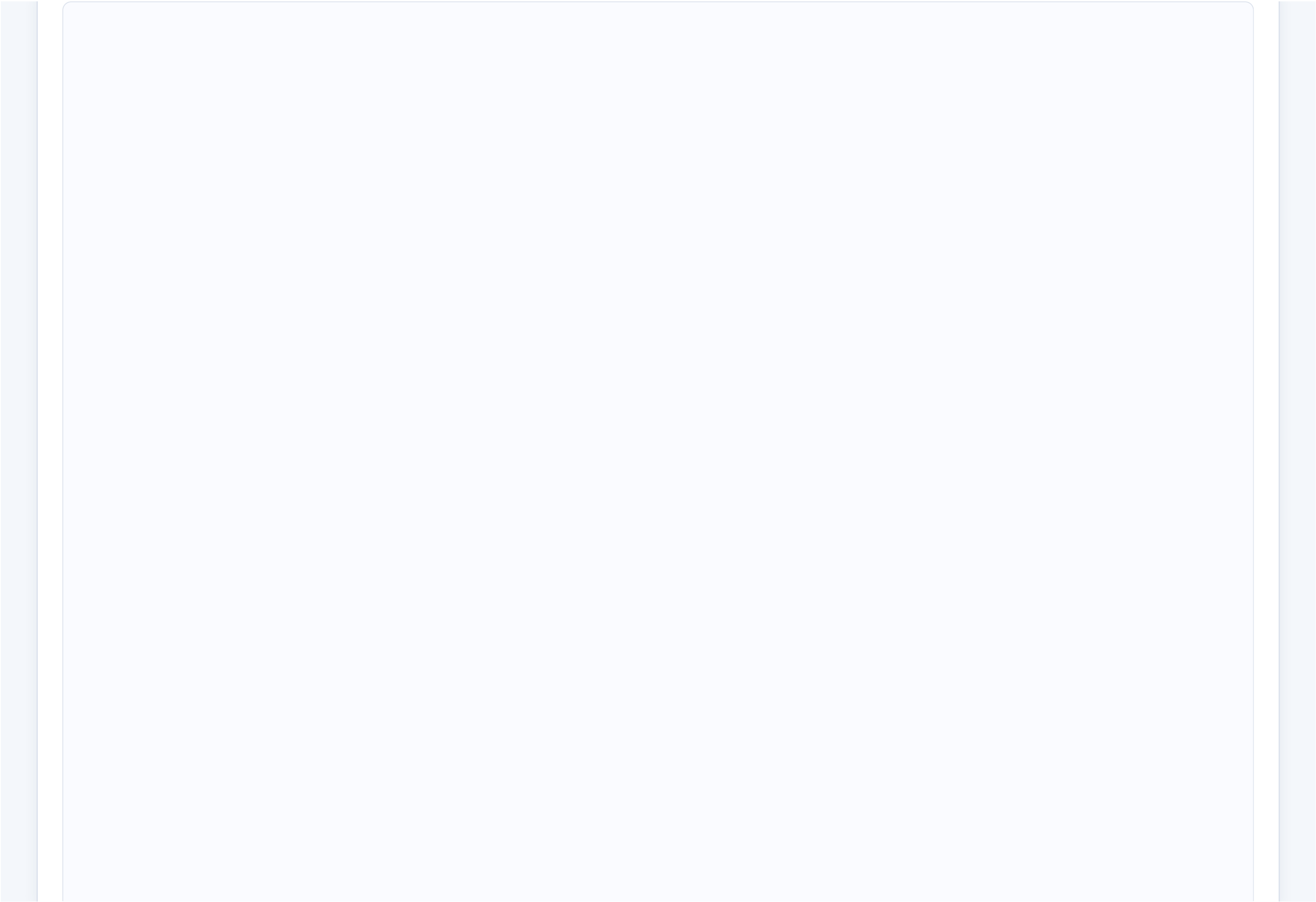




Fig F-24: Account Suspension Detection & Screen

F-25

Password Reset Flow (Forgot Password)

Actor: User (unauthenticated)

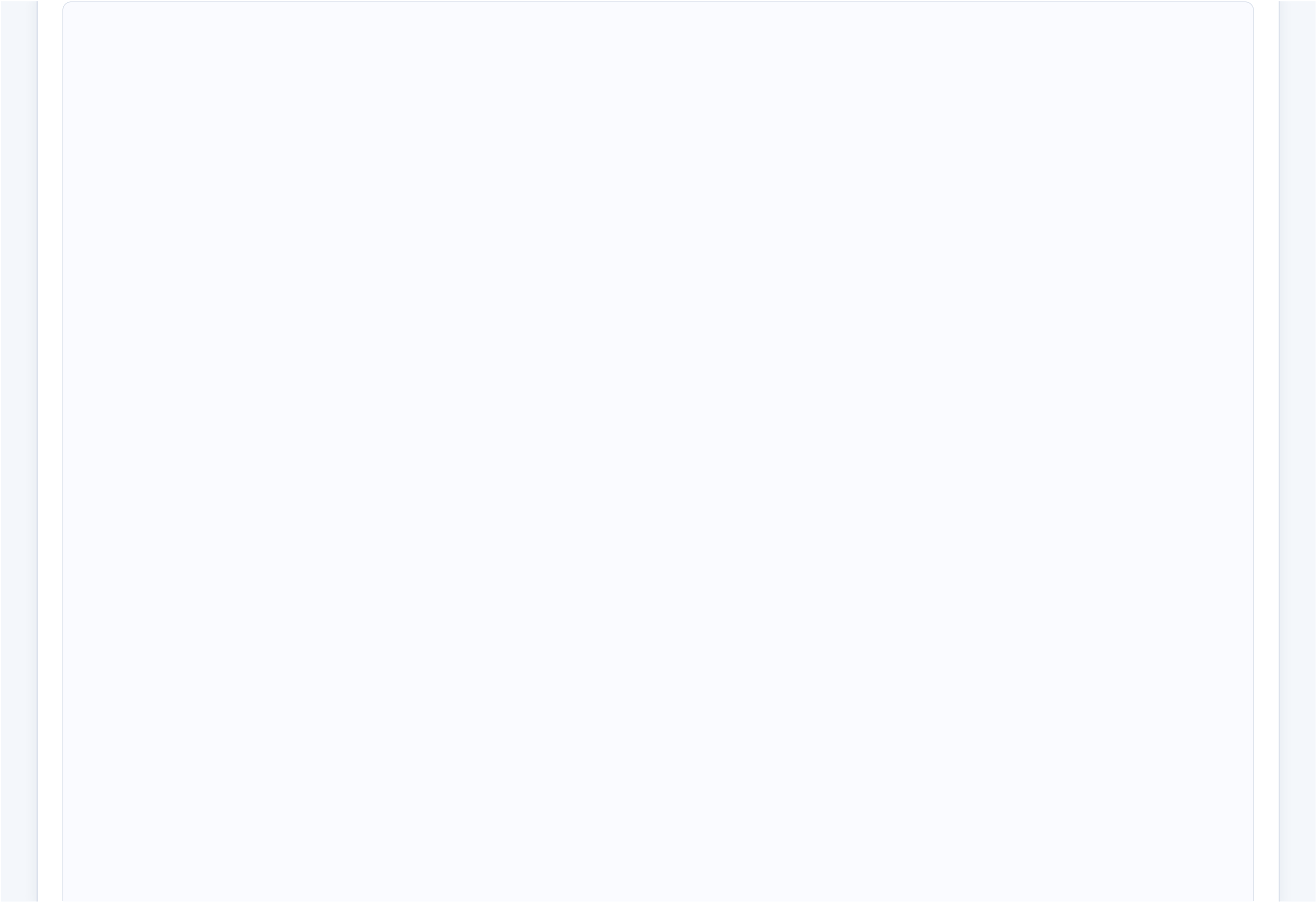
LoginPage

AWS Cognito · SES

All Tiers

A 2-step password recovery flow accessible from the Login page. Step 1 (forgot mode): user enters their email; Cognito sends a verification code via SES. Step 2 (forgotConfirm mode): user enters the code and a new password (min 8 characters). On success the user is redirected to Login. All validation messages are displayed in Hebrew.

Implements: `frontend/src/pages/LoginPage.jsx` — modes `forgot` , `forgotConfirm` AWS Amplify: `resetPassword()` , `confirmResetPassword()`



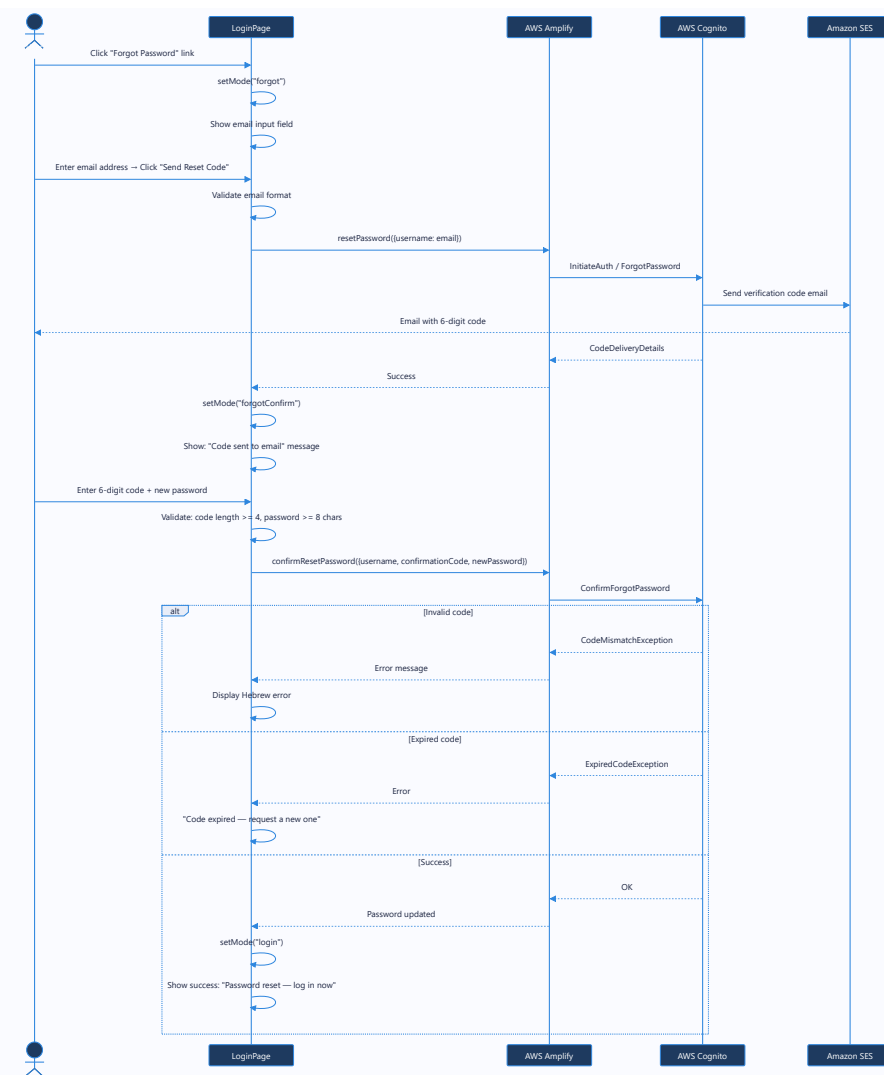


Fig F-25: Forgot Password — 2-Step Reset Flow



Swipe UX — Discovery & Match Insights

F-22 · F-26 · F-27

F-22

Preference Mismatch Warning Modal

Actor: User

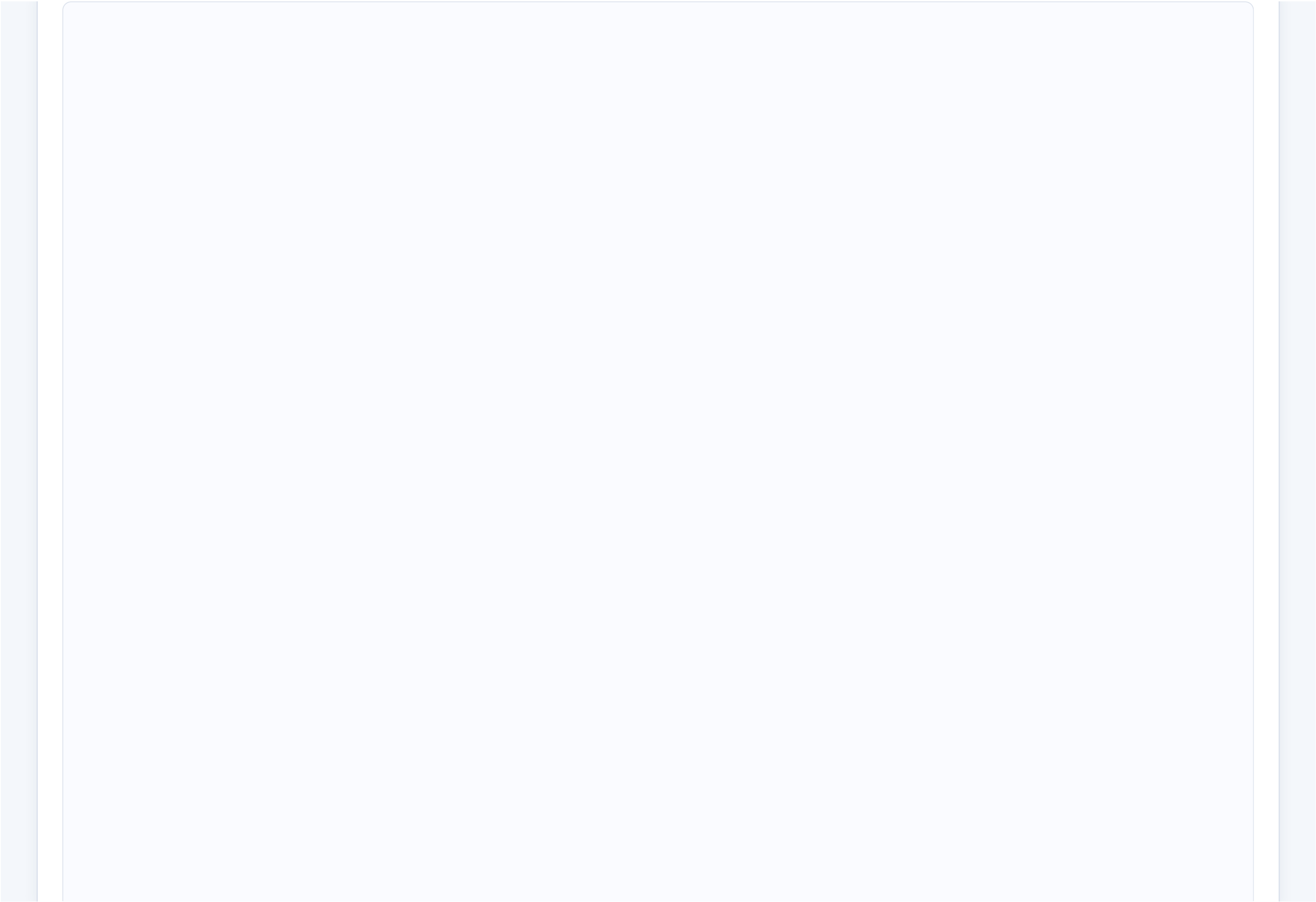
MismatchWarningModal · SwipePage

DynamoDB

All Tiers

When a user attempts to apply to a job flagged as `matchesPreferences: false` (i.e., the job's domain or experience level does not align with the user's profile), an animated warning modal intercepts the action. The modal explains the detected mismatch and offers two choices: cancel or continue anyway. If the user continues, the swipe is recorded normally and AI tailoring may still run.

Implements: `frontend/src/components/MismatchWarningModal.jsx` `frontend/src/pages/SwipePage.jsx` — `matchesPreferences` flag check



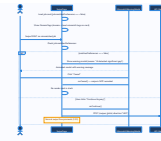


Fig F-22: Preference Mismatch Warning Interception

F-26

Discovery Mode — 30-Minute Temporary Window

Actor: User

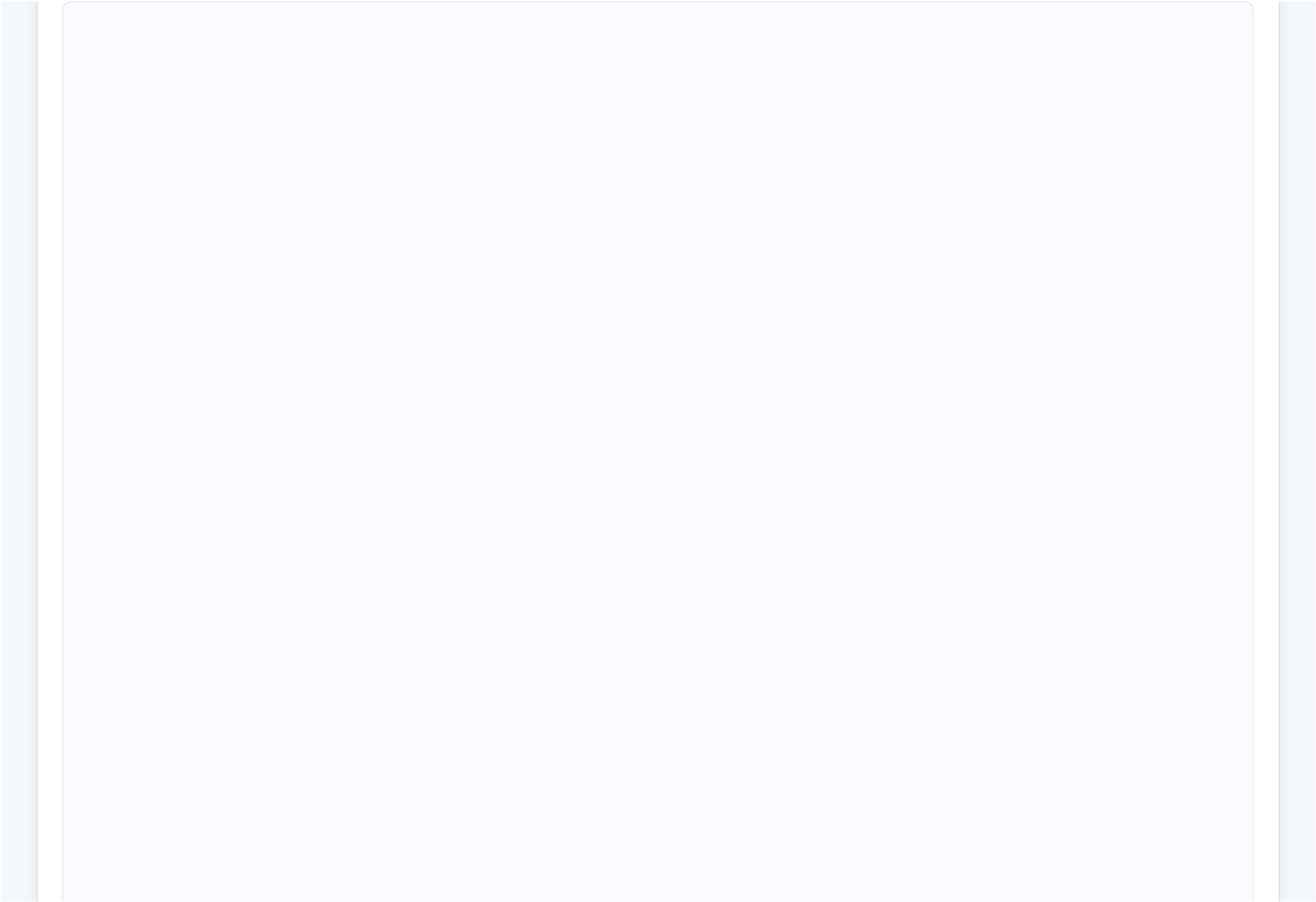
SwipePage

localStorage

All Tiers

When the user has swiped through all their preference-matched jobs but there are still non-matching jobs available, a special "primary deck exhausted" screen appears showing a preview of available domains and a button to activate Discovery Mode for 30 minutes. During this window, all jobs are shown regardless of match. A countdown banner is displayed. The window auto-expires via `setTimeout` and persists across navigation via `localStorage`.

Implements: `frontend/src/pages/SwipePage.jsx` — `discoveryUntil` , `activateDiscovery()` , `DiscoveryPreview`



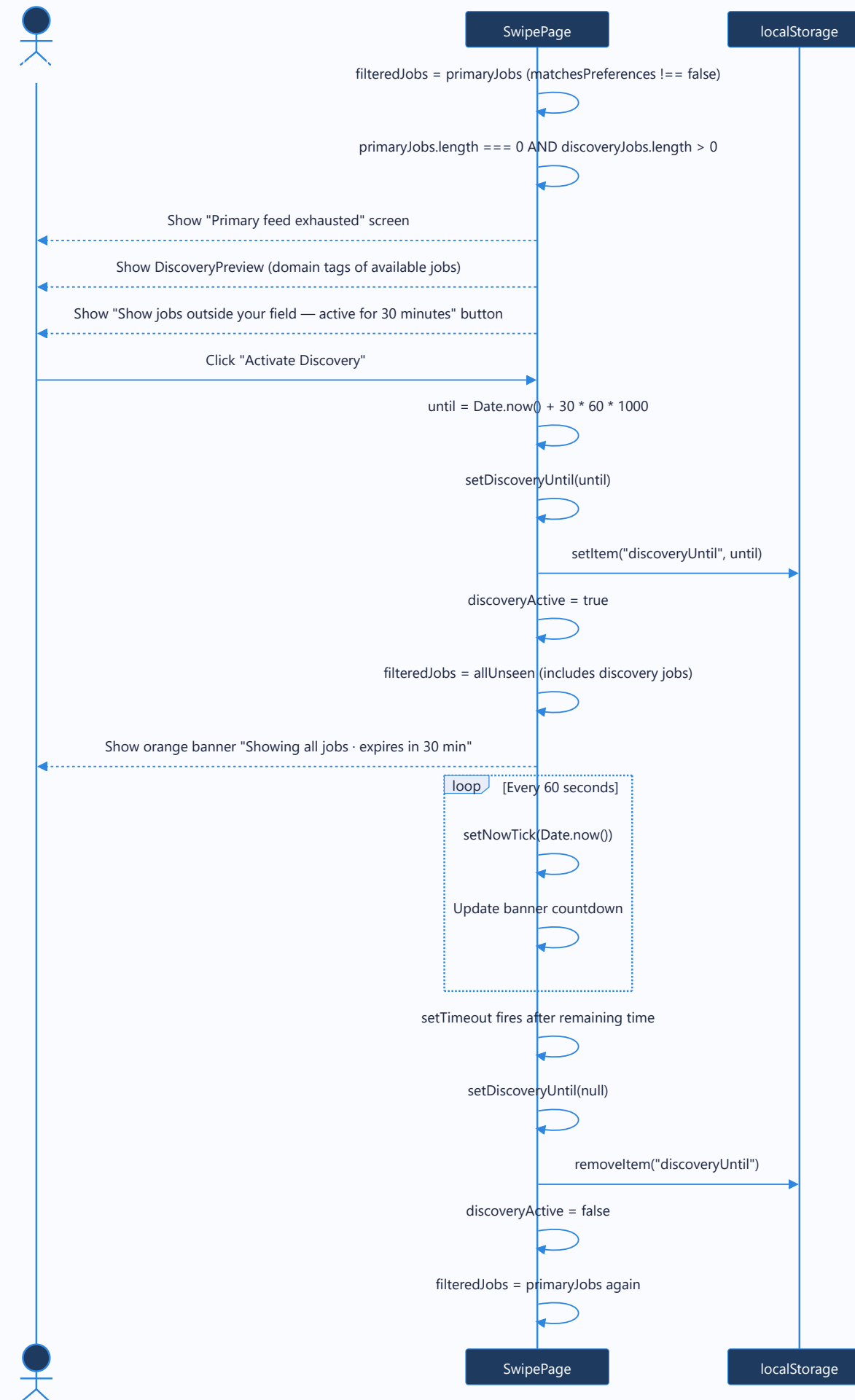


Fig F-26: Discovery Mode — 30-Minute Temporary Window

F-27

Match Score Breakdown Modal

Actor: User

SwipePage · MatchModal

DynamoDB (matchBreakdown)

All Tiers

Each job card displays a coloured match-score badge (e.g. "78% התאמה ⓘ"). Tapping the badge opens an animated modal that breaks the score into three weighted components: Role Match (50 pts), Experience Level (30 pts), Distance (20 pts). Each component shows a colour-coded progress bar, matched/unmatched roles, detected levels, and actionable CV keyword hints to improve the score.

Implements: `frontend/src/pages/SwipePage.jsx` — `MatchModal` , `MatchBadge` components `lambdas/jobs/handler.py` — `matchBreakdown` field in response

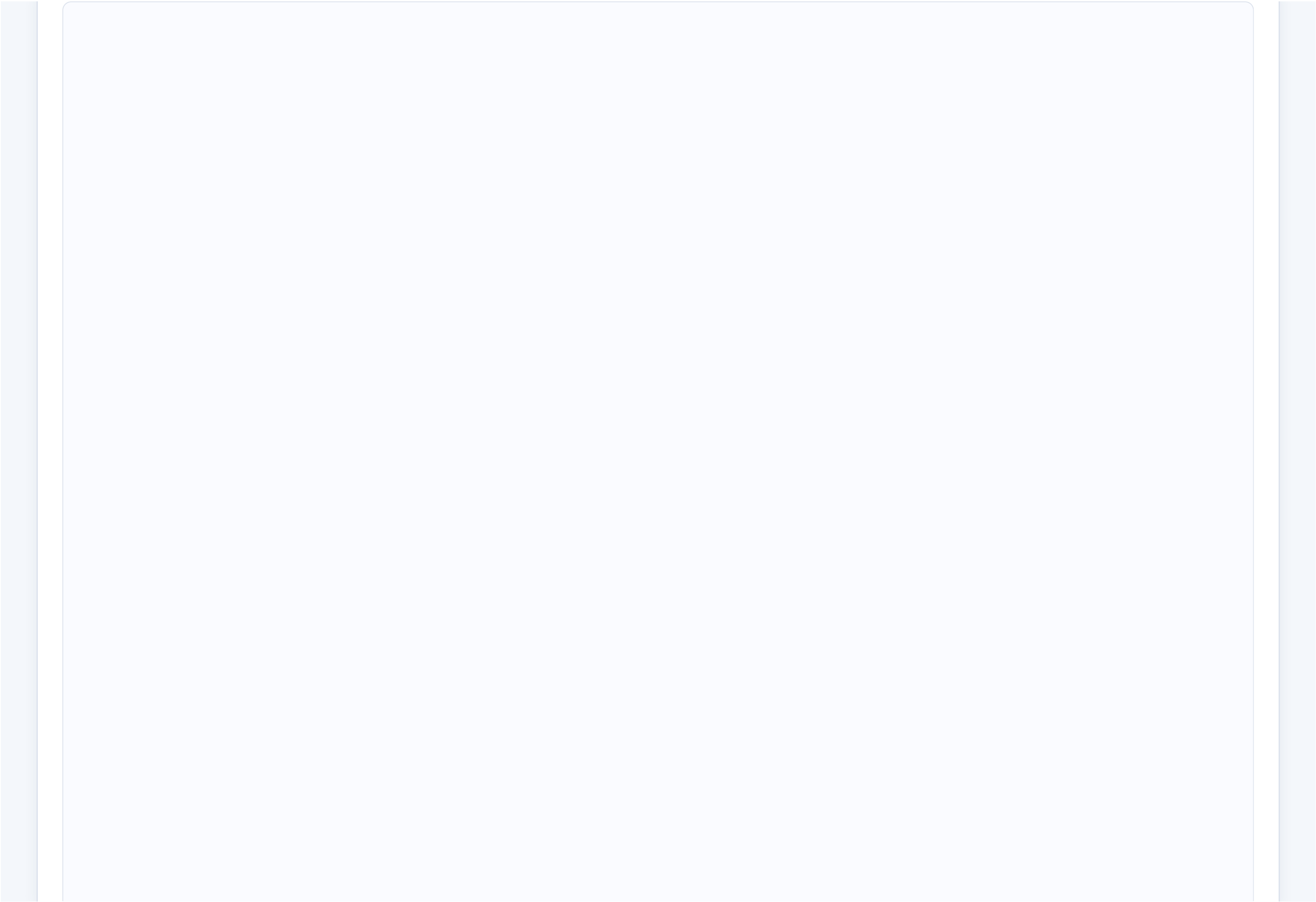




Fig F-27: Match Score Breakdown Modal



Chrome Extension — Advanced Features

F-30 · F-31 · F-32 · F-33

F-30

Chrome Extension — Direct Login (Cognito in Popup)

Actor: User

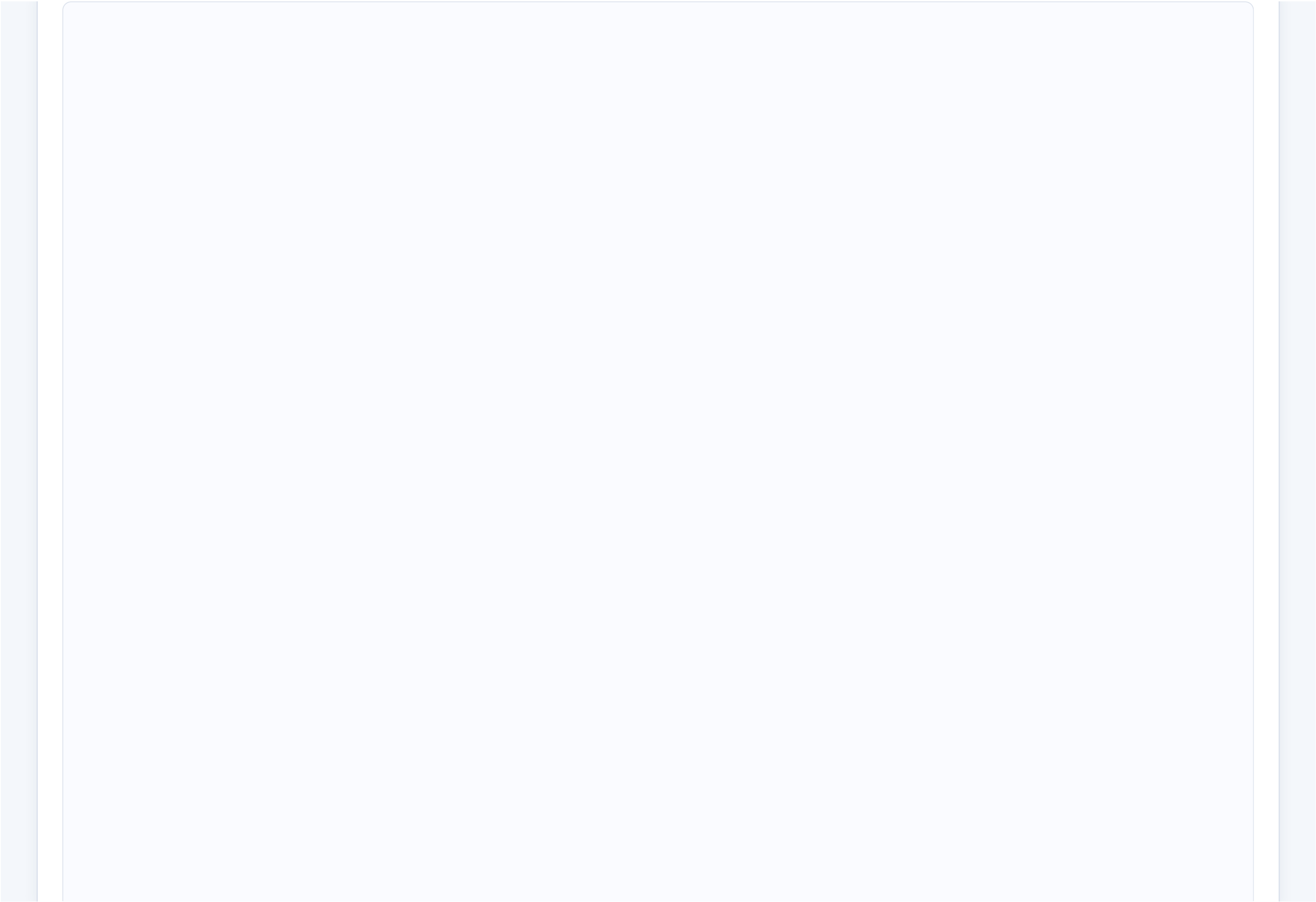
popup.js

AWS Cognito (InitiateAuth)

All Tiers

Users can authenticate the Chrome extension directly inside the popup window without opening a new browser tab. The popup calls Cognito's `InitiateAuth` endpoint with `USER_PASSWORD_AUTH` flow, receives the `idToken`, and stores it in `chrome.storage.local`. Cognito error codes are mapped to localised Hebrew error messages.

Implements: `frontend/chrome-extension/popup.js` — `cognitoLogin()`, `mapCognitoError()`



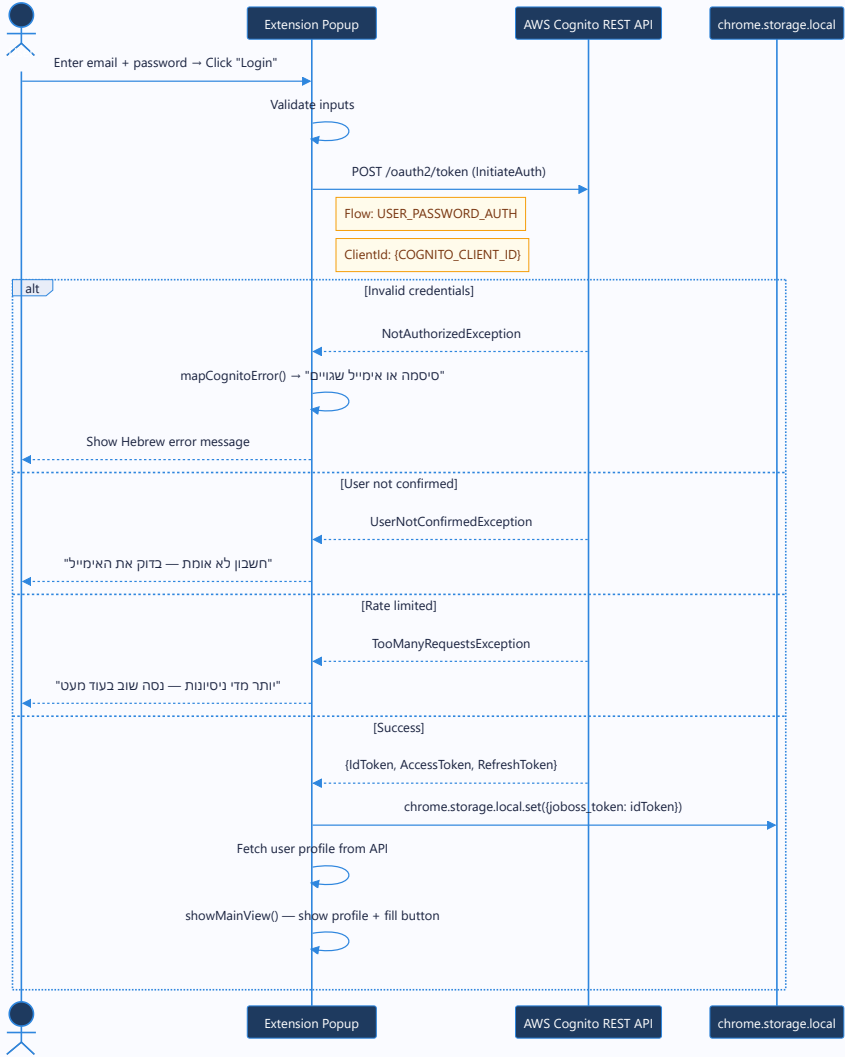


Fig F-30: Chrome Extension Direct Login via Cognito

F-31 Chrome Extension — SSO Auth Bridge

Actor: User (already logged into web app)

popup.js · background.js · AuthExtensionPage

Cognito (token reuse)

All Tiers

As an alternative to direct login (F-30), users who are already authenticated in the JoBoss web app can transfer their session to the extension with a single click. The popup opens the /auth-extension page; that page reads the Cognito idToken from localStorage and sends it back to the extension via `chrome.runtime.sendMessage`. The background service worker receives the message and stores the token.

Implements: `frontend/chrome-extension/popup.js` — SSO button, `getTokenFromTabs()` `frontend/chrome-extension/background.js` — `onMessageExternal` `frontend/src/pages/AuthExtensionPage.jsx`

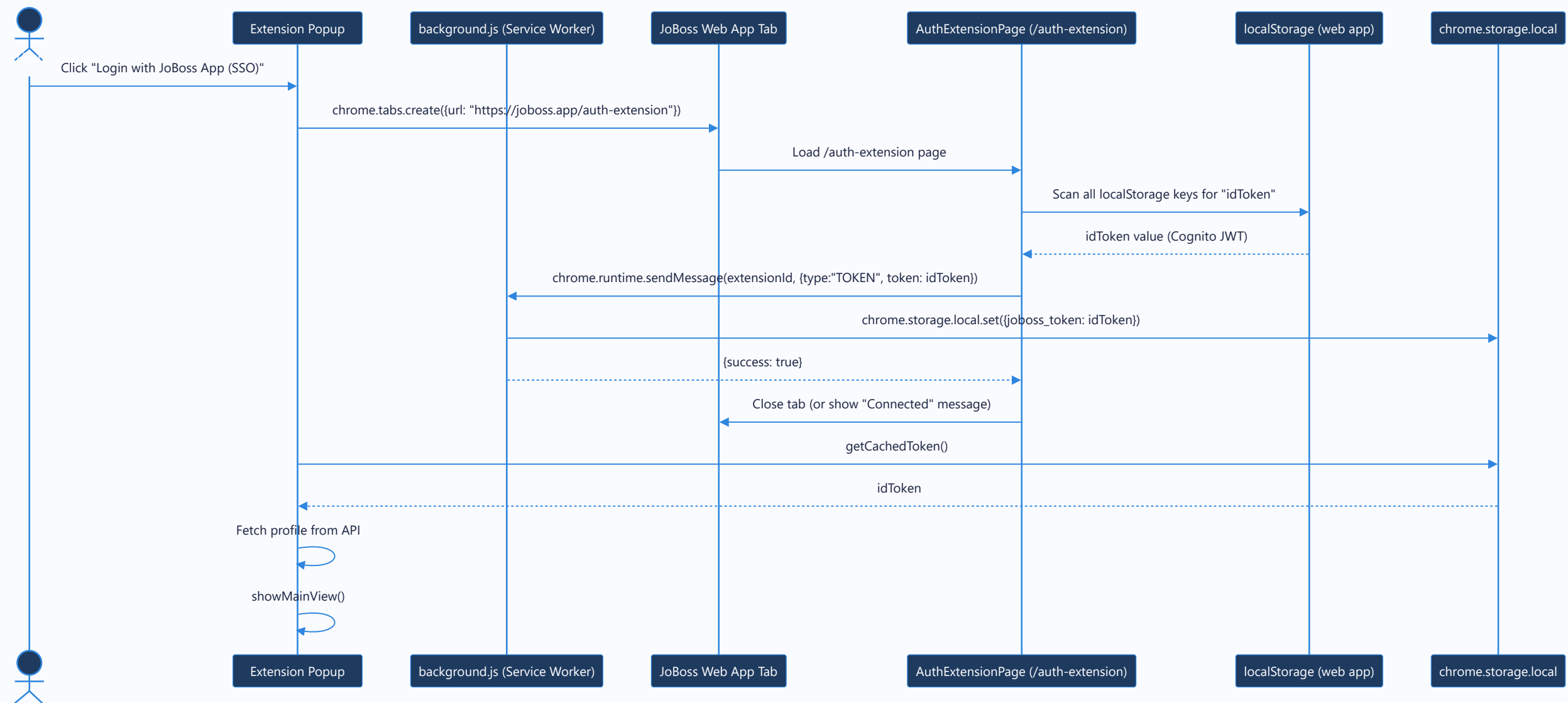


Fig F-31: Chrome Extension SSO Auth Bridge via Web App

F-32

Chrome Extension — Tailored CV Selector

Actor: User (PREMIUM)

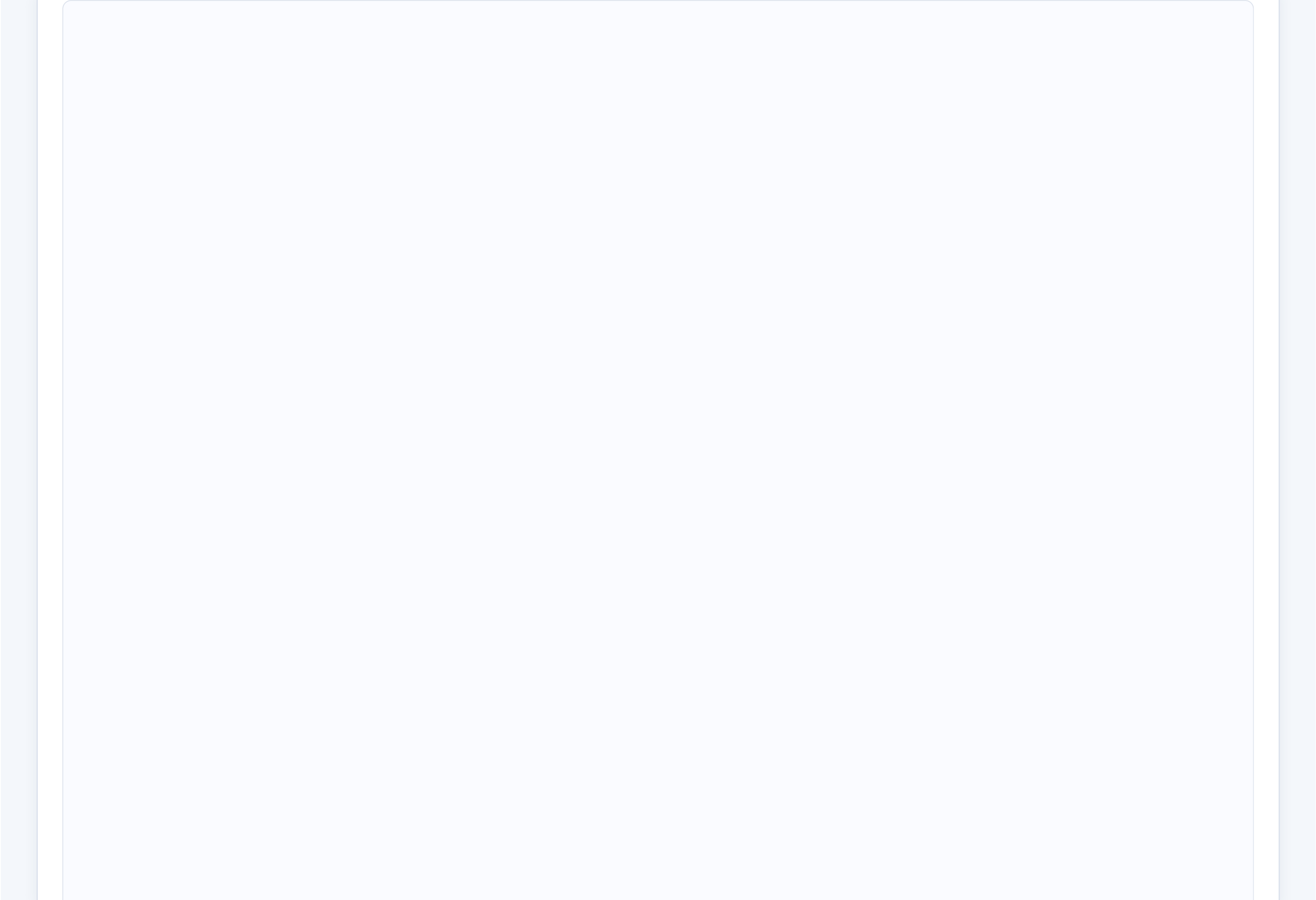
popup.js

S3 Presigned URLs · API Gateway

PREMIUM / PREMIUM_PLUS

When the extension detects that the current page matches a known job application (by exact URL or hostname), it checks if a tailored CV exists for that job in the user's application records. If found, a radio-button selector appears in the popup letting the user choose between their original active resume and the AI-tailored version. The selected presigned S3 URL is then used when the form is auto-filled.

Implements: frontend/chrome-extension/popup.js — loadCVSelector() , loadTailoredCV() lambdas/applications/handler.py — _presign_s3()



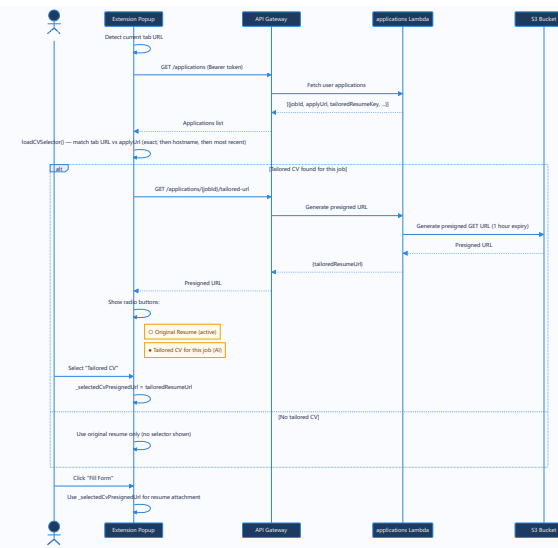


Fig F-32: Chrome Extension Tailored CV Selector

F-33

Chrome Extension — Floating Fill Button (ATS Page Injection)

Actor: User

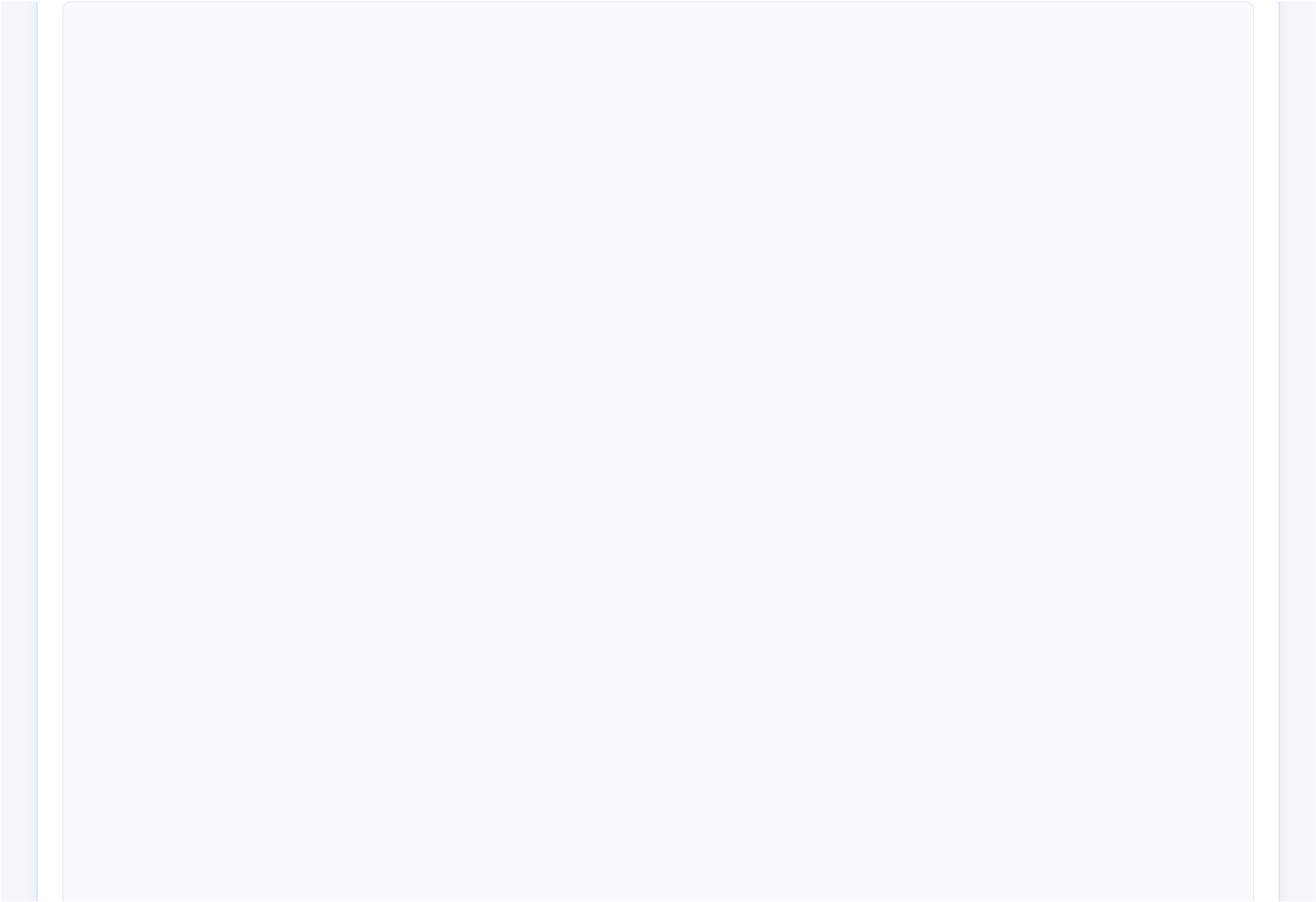
content.js (Content Script)

S3 · DataTransfer API

All Tiers

The content script is injected into 15+ ATS platforms. It scans the page DOM for application form signals (URL keywords, file input, name + email fields). When a form is detected, a floating " ⚡ Fill with JoBoss" button is injected into the page. Clicking it triggers the full auto-fill sequence: field detection by CSS selector patterns, value injection via React-compatible synthetic events, resume attachment via the DataTransfer API, and Hebrew→English city name normalisation.

Implements: frontend/chrome-extension/content.js — hasApplicationForm() , injectFillButton() , fillForm() , attachResumeFromUrl() frontend/chrome-extension/manifest.json — content_scripts matches



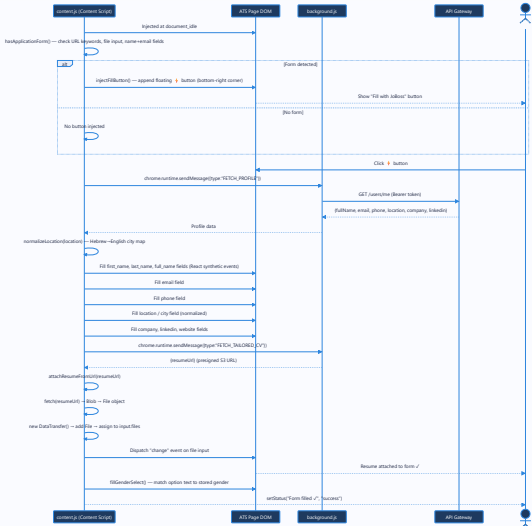


Fig F-33: Chrome Extension Floating Fill Button & Form Injection

Feature Summary Table

ID	Feature Name	Primary Actor	Tier	Key AWS Services
F-01	User Registration & Authentication	User / Visitor	All	Cognito, API Gateway
F-02	Onboarding Flow (5 Steps)	New User	All	Lambda, S3, Bedrock
F-03	Job Discovery & Swipe Interface	User	All	Lambda, DynamoDB
F-04	Job Filtering & Matching Algorithm	System	All	Lambda, DynamoDB
F-05	Swipe Recording (Like / Pass)	User	All	Lambda, DynamoDB, SQS
F-06	Undo Last Swipe	User	All	Lambda, DynamoDB
F-07	Application Tracking (Dual-Track)	User	All	Lambda, DynamoDB
F-08	Auto-Apply Automation	System	PREMIUM+	ECS Fargate, SQS, SES, S3
F-09	AI Resume Tailoring	User	PREMIUM+	Bedrock, S3, Lambda
F-10	Resume Upload & Management	User	All	S3, Lambda, DynamoDB

ID	Feature Name	Primary Actor	Tier	Key AWS Services
F-11	Profile Image Upload & Removal	User	All	S3, Lambda, DynamoDB
F-12	Subscription & Stripe Payment	User	PREMIUM+	Stripe, Lambda, DynamoDB
F-13	Daily Swipe Quota Enforcement	System	All	Lambda, DynamoDB
F-14	Admin Dashboard & Statistics	Admin	Admin	Lambda, DynamoDB, Cognito
F-15	Admin User Management	Admin	Admin	Lambda, DynamoDB, Cognito
F-16	Admin Job Management	Admin	Admin	Lambda, DynamoDB
F-17	Job Importer (Telegram)	System	—	Lambda, Bedrock, DynamoDB, EventBridge
F-18	Job Status Monitoring & Cleanup	System	—	Lambda, DynamoDB, EventBridge
F-19	Chrome Extension Auto-Fill	User	All	S3, API Gateway, Cognito
F-20	Email Notification (SES)	System	PREMIUM+	SES, Lambda
F-21	Personal Dashboard — At-a-Glance Statistics	User	All	Lambda, DynamoDB
F-22	Preference Mismatch Warning Modal	User	All	React (client-side)
F-23	Edit Profile & Change Password	User	All	Cognito, Lambda, DynamoDB
F-24	Account Suspension Screen	System → User	All	Cognito, Lambda, DynamoDB
F-25	Password Reset Flow (Forgot Password)	User (unauthenticated)	All	Cognito, SES
F-26	Discovery Mode — 30-Minute Temporary Window	User	All	localStorage (client-side)
F-27	Match Score Breakdown Modal	User	All	Lambda (matchBreakdown), React
F-28	Show All Jobs Toggle (Permanent Discovery)	User	All	Lambda, DynamoDB, localStorage
F-29	GPS Location Auto-Detection	User	All	Browser Geolocation API, Nominatim
F-30	Chrome Ext. — Direct Login (Cognito in Popup)	User	All	Cognito REST, chrome.storage
F-31	Chrome Ext. — SSO Auth Bridge	User	All	Cognito, chrome.runtime, localStorage
F-32	Chrome Ext. — Tailored CV Selector	User	PREMIUM+	S3 Presigned URLs, Lambda

ID	Feature Name	Primary Actor	Tier	Key AWS Services
F-33	Chrome Ext. — Floating Fill Button (ATS Injection)	User	All	DataTransfer API, S3, API Gateway